

Chapter 7: Thread Synchronization (Intra-Process)

In an ideal threading world, threads would go about their business not bothering other threads. In reality, threads must at times synchronize with each other. The canonical example is accessing a shared data structure, such as a dynamic array. If one thread attempts to insert an item into the array, no other thread should manipulate the same array or even read from it. It might be the case that the threads do so at different times, but since it's all about timing, they may do so at the same time. This would lead to data corruption or some exception. To mitigate that, threads sometimes need to synchronize their work.

Windows provides a rich set of primitives that aid in achieving this (and other) synchronization. In this chapter, we'll examine the synchronization mechanisms available to user mode developers through the Windows API for synchronizing threads within a single process. In the next chapter, we'll look at more synchronization primitives that can be used to synchronize threads running in different processes.

In this chapter:

- **Synchronization Basics**
 - **Atomic Operations**
 - **Critical Sections**
 - **Locks and RAII**
 - **Deadlocks**
 - **The MD5 Calculator Application**
 - **Reader Writer Locks**
 - **Condition Variables**
 - **Waiting on Address**
 - **Synchronization Barriers**
 - **What About the C++ Standard Library?**
-

Synchronization Basics

The classic synchronization is about avoiding a *data race*. A *data race* occurs when two or more threads access the same memory location and at least one of them is writing to that location. Reading concurrently from the same location is never a problem. But once a write enters the picture, all bets are off. Data may become corrupted, reads may become torn (some of the data is read before the change and some after the change). This is where synchronization is required.

In chapter 5, we saw an example application that parallelizes primes numbers calculation. That particular algorithm (fork/join) did not require any synchronization, except for waiting for all threads to complete. This is ideal, as performance can be improved by throwing more CPUs at the problem (at least up to a point). The need to synchronize is not fun, as it reduces performance by definition, since some operations must be performed sequentially rather than concurrently. In fact, the speedup that can be gained by adding more threads/CPU's to a problem depends on the percentage of the code that can be parallelized. This is described nicely by *Amdahl's Law*:

$$\text{Speedup Limit} = \frac{1}{1-p}$$

A more thorough discussion of Amdahl's Law can be found on Wikipedia: https://en.wikipedia.org/wiki/Amdahl%27s_law

Where p is the percentage of code that can be parallelized. For example, if 80% of code can be parallelized, then the maximum speedup possible is 5, no matter how many processors are thrown into the problem.

Most synchronization-related operations require threads to wait upon some condition, until it is safe to proceed, preventing a data race. In the following sections, we'll look at various synchronization options provided by the Windows API, from the simplest to more complex.

Atomic Operations

Some operations that seem so simple and quick are not actually thread safe. Even a simple C variable increment (`x++`) is not thread or multiprocessor safe. Consider, for example, two threads running concurrently on two processors that perform an increment to the same memory location (figure 7-1).

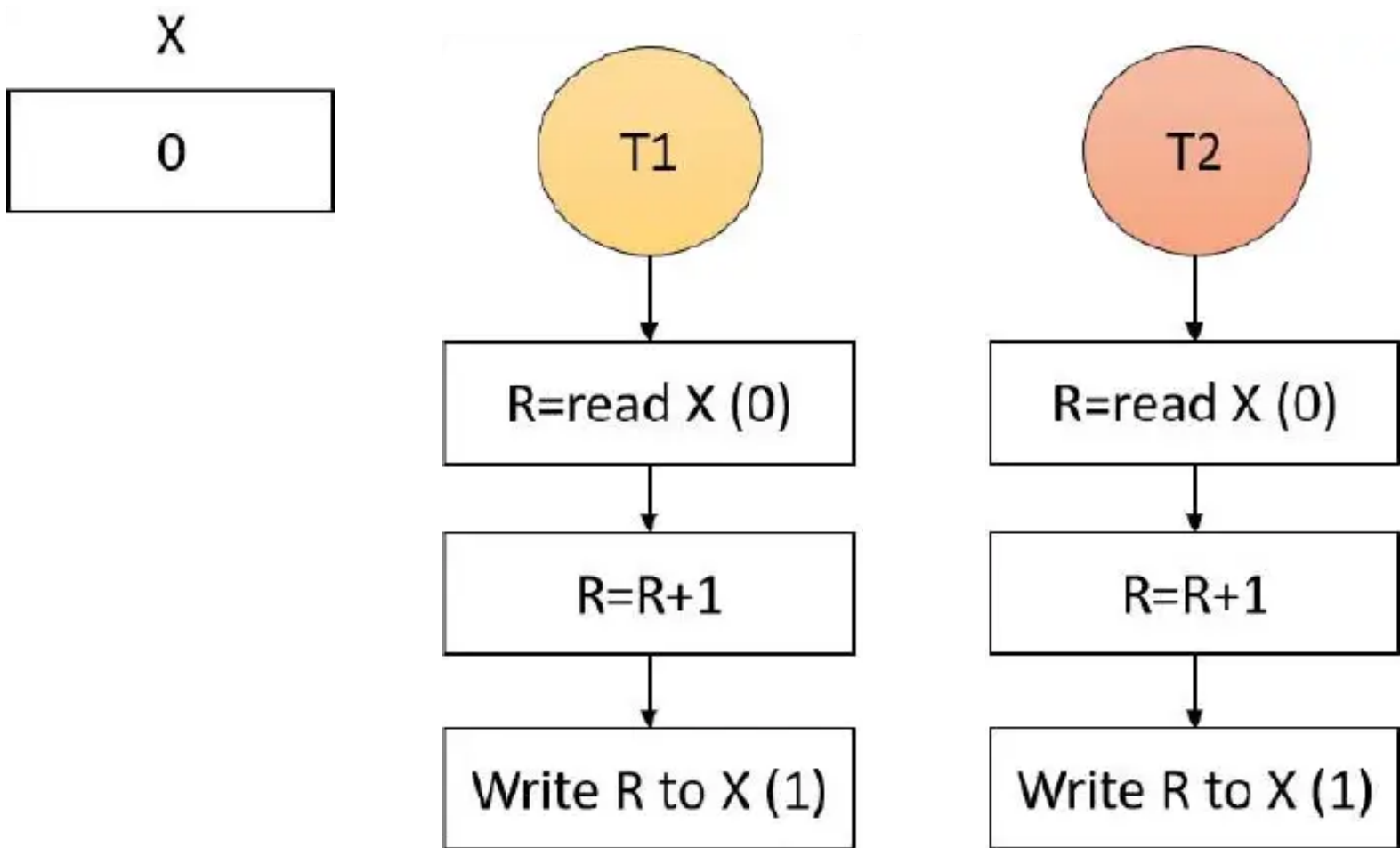


Figure 7-1: Simple increment by multiple threads

Even a simple increment involves a read and write. In figure 7-1, each thread may read the initial value (0) into CPU registers. Each thread increments its processor's register and then writes back the result. The final result written to `X` is 1 instead of 2. This diagram is a gross simplification, as there are other factors at play, such as CPU caches. But even ignoring that, this is clearly a data race. In fact, one of the threads (say `T2`) may be preempted (for example after `R` is incremented), and while `T1` continues incrementing `X`, once `T2` receives CPU time, it writes back 1 to `X`, effectively killing all increments done by thread `T1`.

The Simple Increment Application

The Simple Increment application shown in figure 7-2, uses multiple threads to do one thing: increment a single memory location. The program allows selecting the count of threads to run concurrently and the number of increments each thread should perform. Once the *Run* button is clicked, operations are on the way. When done, the actual result and the expected result, along with the time it took to execute.

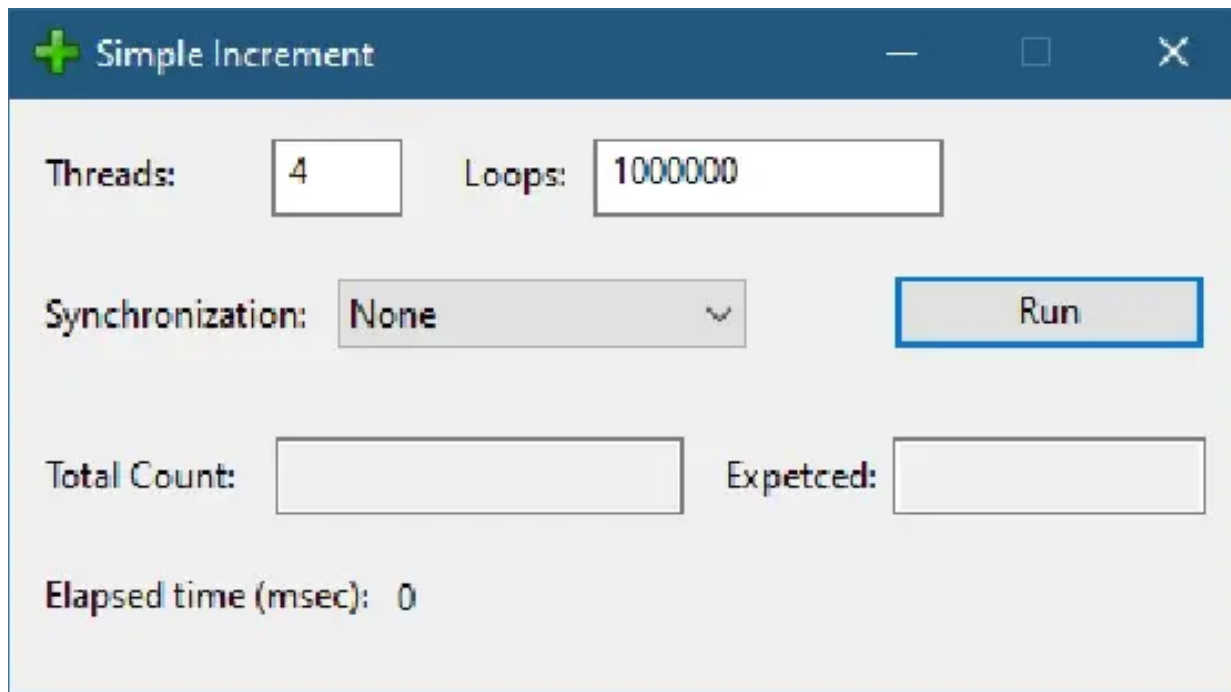
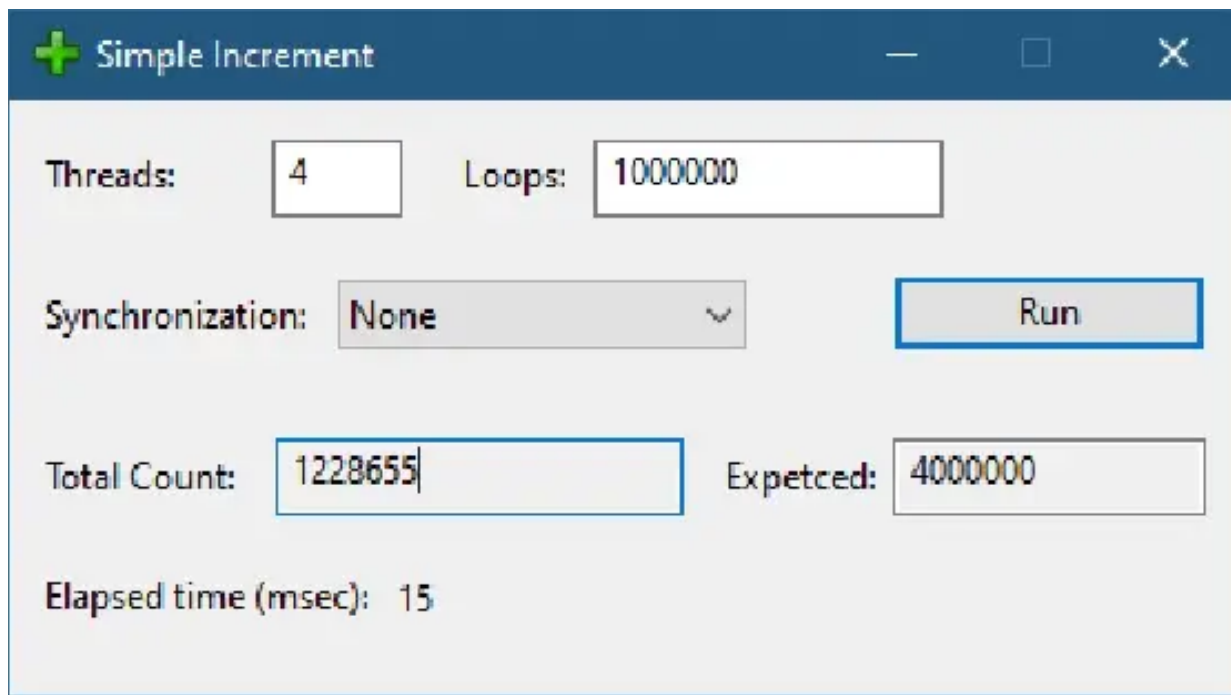


Figure 7-2: The *Simple Increment* application

The *Synchronization* combobox allows selecting how to synchronize the increments. The first (and default) option ("None") is to simply use the ++ operator on the shared memory location - no synchronization at all. Clicking *Run* with the default options shows something like figure 7-3.

Figure 7-3: *Simple Increment* application with no synchronization

Notice the final result is nowhere near the expected result. This is because of the lack of synchronization, causing increments to become “lost”. You can click *Run* again and you’ll get a different result. This is the nature of synchronization issues. The following is the piece of code responsible for doing the multi-threaded increments with a simple ++ operations (in *MainDlg.cpp*):

```
void CMainDlg::DoSimpleCount() {
    auto handles = std::make_unique<HANDLE[]>(m_Threads);
    for (int i = 0; i < m_Threads; i++) {
        handles[i] = ::CreateThread(nullptr, 0, [](auto param) {
            return ((CMainDlg*)param)->IncSimpleThread();
        }, this, 0, nullptr);
    }
    ::WaitForMultipleObjects(m_Threads, handles.get(), TRUE, INFINITE);
    for (int i = 0; i < m_Threads; i++)
        ::CloseHandle(handles[i]);
}

DWORD CMainDlg::IncSimpleThread() {
    for (int i = 0; i < m_Loops; i++)
        m_Count++;
    return 0;
}
```

As we've seen in chapter 5, passing information to a thread is done via the `PVOID` parameter of `CreateThread`. However, in many cases it's more convenient to have the thread function be an instance function, rather than static or global. A useful trick is to pass this as the parameter and use it to call into an instance function, where the entire object state is available. This allows the `IncSimpleThread` function to be an instance one, rather than static.



You may be wondering why not just capture the `this` pointer and use the data members directly? Unfortunately, API functions can only use non-capturing lambda functions. This is why this trick is needed.

In the above code, `m_Threads` is the number of threads, `m_Loops` is the number of iterations to do and `m_Count` is the shared memory location being incremented.

This is clearly an artificial example, where millions of increments are performed on the same memory location, which exposes the bug easily. In real applications, these increments are much less frequent, which means any synchronization bugs are less likely to occur, and in fact may be missed by developers and QA, only to be discovered on customer's machines.

The Interlocked Family of Functions

The solution to the above synchronization issue is in performing the increment as an *atomic operation*, so that any increment is isolated from other increments and any other access to the same memory location using other Interlocked functions. This atomic operation and other similar operations are exposed in the Windows API through a set of functions with the Interlocked prefix. In the simple increment case, that's `InterlockedIncrement`:

```
unsigned InterlockedIncrement(unsigned volatile *Addend);
```

This performs an atomic increment, and as a bonus, returns the new value in addition to actually changing the memory location. Behind the covers, this is not a true function, but rather a *compiler intrinsic* that issues a special instruction to the CPU to perform this operation atomically. This is great, since leveraging the hardware is always going to be faster than software. Also, since there is no explicit “lock” object used, no deadlock is possible with these functions.

Back to the *Simple Increment* application, the second synchronization method in the combobox sets the increment method to `InterlockedIncrement`, used in the `InterlockedThread` function:

```
DWORD CMainDlg::IncInterlockedThread() {  
    for (int i = 0; i < m_Loops; i++)  
        ::InterlockedIncrement((unsigned*)&m_Count);  
    return 0;  
}
```

Figure 7-4 shows an example run with the *Interlocked* option selected in the synchronization combobox.

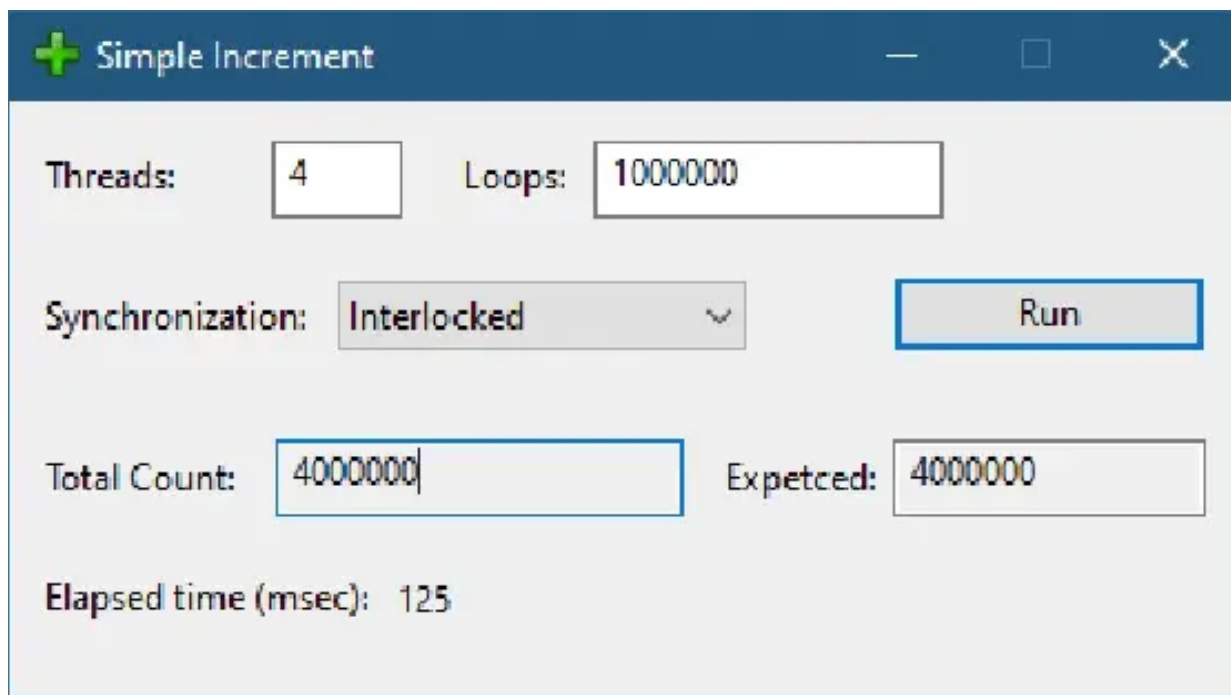


Figure 7-4: *Simple Increment* application with *Interlocked* synchronization

Other simple functions in the same vein include `InterlockedDecrement`, `InterlockedAdd`, `InterlockedExchange`, `InterlockedAnd`, `InterlockedOr`, `InterlockedXor`, `InterlockedExchangePointer`, `InterlockedCompareExchange` and a few others. These exist for 64 bit and 16 bit values as well, with the suffixes 64 and 16, respectively, added to the function names (e.g. `InterlockedIncrement64`).



There are also extended functions, such as `InterlockedAndAcquire`, `InterlockedAndRelease` and `InterlockedAndNoFence` (and similar for other operations). These specialized versions also specify the acquire/release semantics on the memory location. Discussing these variants is out of scope for this book, and you should use the “standard” functions, which are the safest, unless you know what you’re doing. More information on fences and acquire/release semantics can be found on the web. One of the best is the talk “[Atomic<> Weapons](https://channel9.msdn.com/Shows/Going+Deep/Cpp-and-Beyond-2012-Herb-Sutter-atomic-Weapons-1-of-2)”¹ by Herb Sutter. You can also (or alternatively) watch my (somewhat abridged) session “[Concurrency and the C++ Memory Model](https://www.youtube.com/watch?v=NZ_ncor_Lj0)”².

The `InterlockedCompareExchange` function is mostly used when doing *lock free programming*, a paradigm that uses CPU intrinsics to avoid using any locks in software. This topic is out of scope for this book, as it’s not Windows-specific. However, the Windows API offers a lock-free, singly linked list implementation. These functions use the `SLIST_HEADER` union as a linked list header and `SLIST_ENTRY` structures as the entries that can be added/removed to/from the list, atomically.

Both types are fully defined in the SDK headers, but only `SLIST_ENTRY` should be marginally interesting to look at:

```
typedef struct DECLSPEC_ALIGN(16) _SLIST_ENTRY {
    struct _SLIST_ENTRY *Next;
} SLIST_ENTRY, *PSLIST_ENTRY;
```



`SLIST_ENTRY` and `SLIST_HEADER` must be aligned on a 16 byte boundary, indicated by decorating the types with the `__declspec(align(16))` VC++ compiler attribute. Stack-based or static allocation of these types will work correctly, but more often than not you’ll need to dynamically allocate `SLIST_ENTRY`. The C runtime offers the `_aligned_malloc` function that guarantees a specified alignment of a memory allocation.

Clearly, it’s a classic single list entry item. But where is the actual data? The expectation is that your data item includes as a first entry `SLIST_ENTRY` itself. This ensures the alignment requirement is met for the `SLIST_ENTRY`. The following example shows a data item type suitable to be stored in the described linked list:

¹<https://channel9.msdn.com/Shows/Going+Deep/Cpp-and-Beyond-2012-Herb-Sutter-atomic-Weapons-1-of-2>

²https://www.youtube.com/watch?v=NZ_ncor_Lj0

```

struct MyDataItem {
    SLIST_ENTRY Entry;
    int MyValue;
    //...
};

```

Since the operations on the list must be lock-free, and the list is singly linked, it actually implements a stack. There is no thread-safe way to add items at the tail of the list. This is why the main operations on the list use the names “Push” and “Pop” - terms used in stack-based data structures.

Table 7-1 lists the functions available for lock-free singly linked list manipulation.

Table 7-1: Functions for working with singly linked lists

Function	Description
InitializeSListHead	Initializes the list’s head to an empty list.
InterlockedPushEntrySList	Inserts an item in the front of the list.
InterlockedPopEntrySList	Removes the item in the front of the list.
InterlockedPushListSListEx	Inserts multiple items in from of the list.
InterlockedFlushSList	Removes all items from the list, returning the item that was in front (if any).
QueryDepthSList	Returns the count of items in the list. This function is not thread safe and is best avoided. It’s much better to keep track of the item count yourself (with InterlockedIncrement / InterlockedDecrement).

Critical Sections

The Interlocked family of functions is great with simple cases such as integer increments. However, for other operations, a more general mechanism is required. A *critical section* is a classic synchronization mechanism based on one thread at most acquiring a lock. Once a thread acquired a specific lock, no other thread can acquire the same lock until the thread that acquired it in the first place releases it. Only then, one (and only one) of the waiting threads can acquire the lock. This means that at any given moment, no more than one thread has acquired the lock. This idea is depicted in figure 7-5.

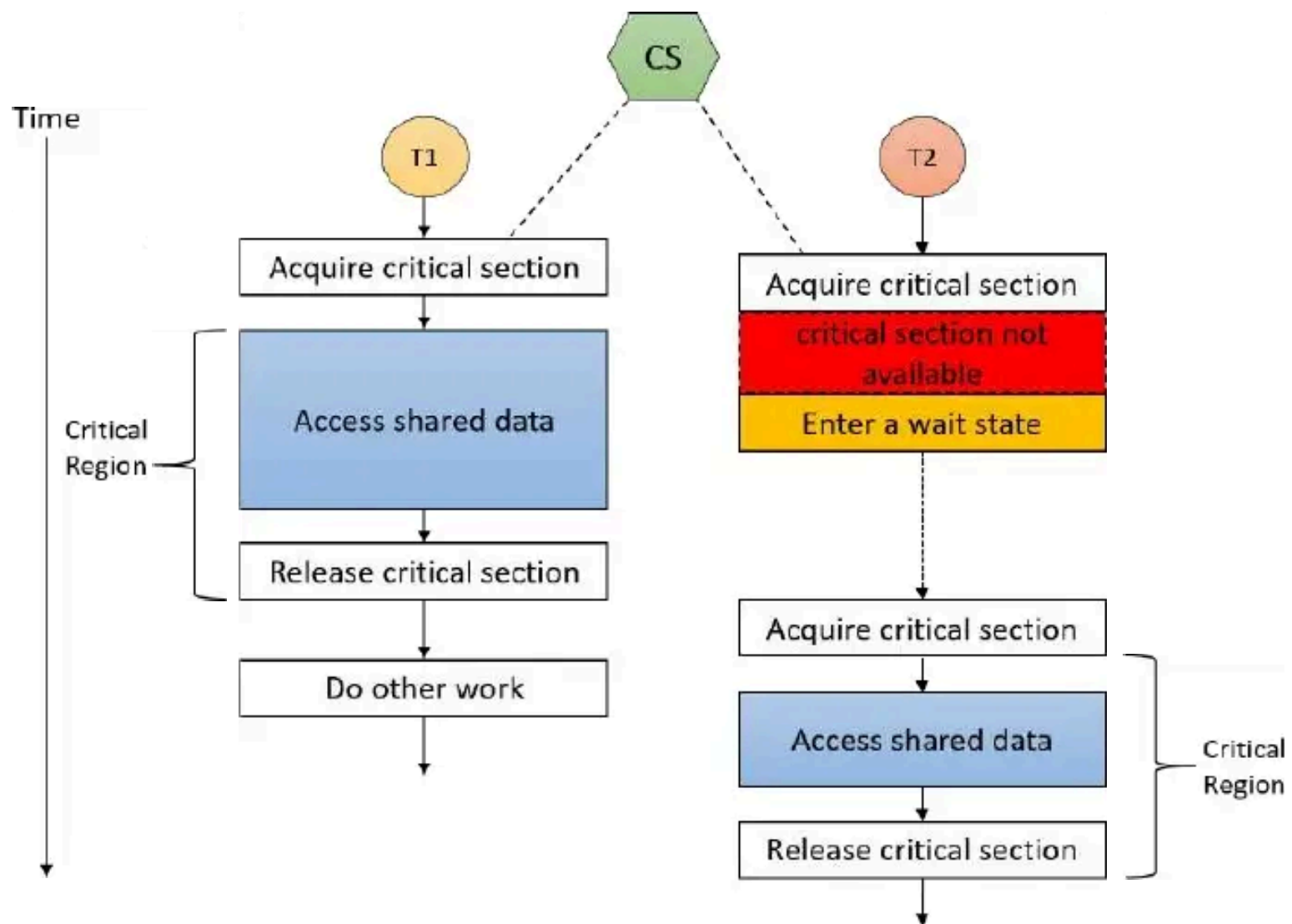


Figure 7-5: Synchronization with a critical section

The thread that acquired the lock is also its *owner*, which means two things:

1. The owner thread is the only thread that can release the critical section.
2. If the owner thread attempts to acquire the critical section a second time (recursively), it succeeds automatically, incrementing an internal counter. This means the owner thread now has to release the critical section the same number of times to truly release it.

The code between acquire and release of the lock is called a *critical region*.

The critical section itself is represented by the `CRITICAL_SECTION` structure, in itself a typedef to another structure, `RTL_CRITICAL_SECTION`. Although the structure is fully

defined, you should treat it as opaque. Initializing the critical section must be done with one of the following functions:

```
void InitializeCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

```
BOOL InitializeCriticalSectionAndSpinCount(  
    LPCRITICAL_SECTION lpCriticalSection,  
    DWORD dwSpinCount);
```

```
BOOL InitializeCriticalSectionEx(  
    LPCRITICAL_SECTION lpCriticalSection,  
    DWORD dwSpinCount,  
    DWORD Flags);
```

The initialization of a critical section involves setting its members to some initial values. This is why the first function returns *void*.

The second and third variants set a *spin count* for the critical section. The idea is that if a critical section cannot be acquired by a thread, it should enter a wait state because some other thread holds the critical section. However, entering a wait state requires the thread to transition to kernel mode, which is not cheap. A compromise could be to spin a small amount of time because it's likely that the current owner of the critical section will release it very soon, and so a transition to the kernel can be avoided. The spin count maximum value is 0x00ffffff (the remaining hex digits are used internally as flags).

What should the spin count be? It's difficult to answer that categorically, because it depends on the actual processor type and other hardware factors. The default spin count is 2000 (used by `InitializeCriticalSection`).



If the system has a single processor (or the process image file has the “Single CPU” flag in its PE header), the spin count is always set to zero. This makes sense, since another thread would never be able to release the critical section while this thread is spinning, since there are no more processors.

The last initialization function adds a flags parameter. Several are defined in the headers, but only one is documented - `CRITICAL_SECTION_NO_DEBUG_INFO` (0x01000000), which specifies that the critical section structure should not allocate an extra debug structure that can help with diagnosing critical section issues.

When the critical section is not needed anymore, call `DeleteCriticalSection`:

```
void DeleteCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

Once the critical section is initialized, it can be acquired and released by threads using the following functions:

```
void EnterCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

```
void LeaveCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

EnterCriticalSection attempts to acquire the critical section and only returns when it does. If the calling thread is already the owner of the critical section, it continues immediately. Conversely, LeaveCriticalSection releases an already acquired critical section.



Curiously enough, any thread can call LeaveCriticalSection (not just the current owner thread) and succeed. I would expect the function to throw an exception (since it returns void). But it just releases the critical section, turning the owner thread ID back to zero.

A simple example is shown in the *Simple Increment* application. If you select the synchronization mechanism as “Critical Section” in the combobox and click *Run*, the increments would now be protected by a critical section (figure 7-6).

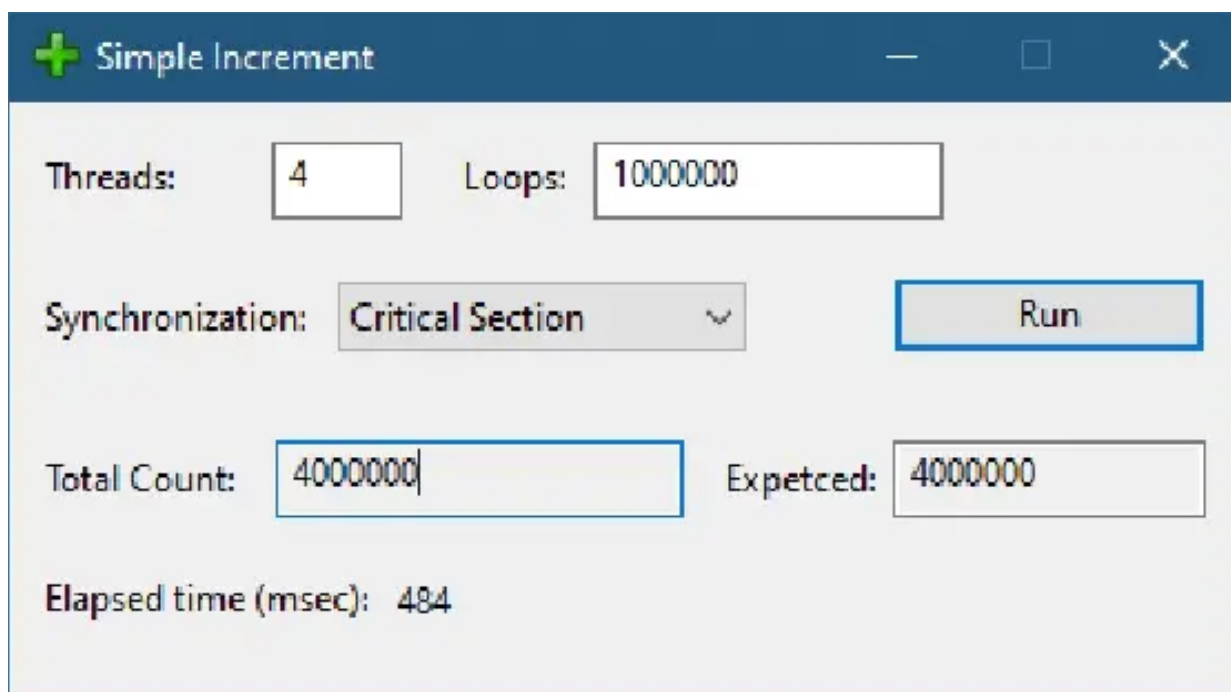


Figure 7-6: *Simple Increment* application with critical section synchronization

The code to accomplish this synchronization in *Simple Increment* is the following:

```
// m_CritSection is CRITICAL_SECTION
```

```
void CMainDlg::DoCriticalSectionCount() {
    auto handles = std::make_unique<HANDLE[]>(m_Threads);
    ::InitializeCriticalSection(&m_CritSection);

    for (int i = 0; i < m_Threads; i++) {
        handles[i] = ::CreateThread(nullptr, 0, [] (auto param) {
            return ((CMainDlg*)param)->IncCriticalSectionThread();
        }, this, 0, nullptr);
    }
    ::WaitForMultipleObjects(m_Threads, handles.get(), TRUE, INFINITE);
    for (int i = 0; i < m_Threads; i++)
        ::CloseHandle(handles[i]);
    ::DeleteCriticalSection(&m_CritSection);
}

DWORD CMainDlg::IncCriticalSectionThread() {
    for (int i = 0; i < m_Loops; i++) {
        ::EnterCriticalSection(&m_CritSection);
        m_Count++;
        ::LeaveCriticalSection(&m_CritSection);
    }
    return 0;
}
```

Each call to `EnterCriticalSection` must be matched by a `LeaveCriticalSection` in the same function. It's too dangerous to call some other function within the critical region that is expected to call `LeaveCriticalSection`. To avoid possible bugs, always use these pair of functions within the same

The `EnterCriticalSection` waits for the critical section to be available as long as it takes. There is no way to specify a timeout, but there is a way to inspect the critical section, if it's free - acquire it; otherwise, continue execution. This is exactly what `TryEnterCriticalSection` does:

```
BOOL TryEnterCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

Locks and RAII

As we've seen in the previous section, `EnterCriticalSection` and `LeaveCriticalSection` are natural pairs. It would be unfortunate to “forget” calling `LeaveCriticalSection`, for example by returning from the function before the call. This mistake is easy to make, and even if there is no such bug, it forces the developer to think about it and make sure that any future modifications to that function don't break the pair of calls.

It would be much better to have code that automatically calls `LeaveCriticalSection` no matter what, without the code having to worry about it. There are two ways to get this behavior: termination handlers and C++ *Resource Acquisition Is Initialization* (RAII).

Termination handlers will be discussed in more detail in a future chapter, but here are the essentials:

```
CRITICAL_SECTION cs;

void DoWork() {
    ::EnterCriticalSection(&cs);
    __try {
        // manipulate shared resource
    }
    __finally {
        ::LeaveCriticalSection(&cs);
    }
}
```

The `__try` and `__finally` are two Microsoft-specific keywords extending the C language for the sake of running the code in the `__finally` block when leaving the `__try` block, no matter what. Even a `return` statement within the `__try` block would first call the `__finally` block and only then actually return from the function.

If you're using C (and not C++), then termination handlers are your best option. When working with C++, it's better (and more convenient) to leverage constructors and destructors that can execute code automatically when objects are constructed and destroyed, respectively. (This is known as the RAII idiom in C++)

For a critical section, a RAII class could look something like this:

```
// AutoCriticalSection.h
```

```
struct AutoCriticalSection {
    AutoCriticalSection(CRITICAL_SECTION& cs);
    ~AutoCriticalSection();

    // delete copy ctor, move ctor, assignment operators
    AutoCriticalSection(const AutoCriticalSection&) = delete;
    AutoCriticalSection& operator=(const AutoCriticalSection&) = delete;
    AutoCriticalSection(AutoCriticalSection&&) = delete;
    AutoCriticalSection& operator=(AutoCriticalSection&&) = delete;

private:
    CRITICAL_SECTION& _cs;
};
```

```
// AutoCriticalSection.cpp
```

```
AutoCriticalSection::AutoCriticalSection(CRITICAL_SECTION& cs) : _cs(cs) {
    ::EnterCriticalSection(&_cs);
}

AutoCriticalSection::~AutoCriticalSection() {
    ::LeaveCriticalSection(&_cs);
}
```

The code is part of the *ThreadingHelpers* project in this chapter's code samples.

The constructor acquires the critical section and the destructor releases it. Using it in the *Simple Increment* application could be done like so:

```

DWORD CMainDlg::IncCriticalSectionThread() {
    for (int i = 0; i < m_Loops; i++) {
        AutoCriticalSection locker(m_CritSection);
        m_Count++;
    }
    return 0;
}

```

While we're on the subject of RAII, it may be a good idea to wrap the critical section itself in a RAII class, so that critical section initialization and deletion is automatic as well. Here is a possible implementation:

// CriticalSection.h

```

class CriticalSection : public CRITICAL_SECTION {
public:
    CriticalSection(DWORD spinCount = 0, DWORD flags = 0);
    ~CriticalSection();

    void Lock();
    void Unlock();
    bool TryLock();
};

```

// CriticalSection.cpp

```

CriticalSection::CriticalSection(DWORD spinCount, DWORD flags) {
    ::InitializeCriticalSectionEx(this, (DWORD)spinCount, flags);
}

CriticalSection::~CriticalSection() {
    ::DeleteCriticalSection(this);
}

void CriticalSection::Lock() {
    ::EnterCriticalSection(this);
}

void CriticalSection::Unlock() {

```

```

        ::LeaveCriticalSection(this);
    }

    bool CriticalSection::TryLock() {
        return ::TryEnterCriticalSection(this);
    }

```

The `Lock`, `Unlock` and `TryLock` are not required, but may be helpful in some scenarios.

The derivation from `CRITICAL_SECTION` allows passing `CriticalSection` if `CRITICAL_SECTION` is required. Alternatively, a `CRITICAL_SECTION` member could be embedded inside the structure, with an operator that can implicitly cast `CriticalSection` to `CRITICAL_SECTION`. I leave this implementation as an exercise for the reader.

Deadlocks

Working with critical sections seems simple enough. Even if we work with the various RAI wrappers, there is still a danger of deadlocks. A classic deadlock occurs when thread A that owns lock 1 (e.g. a critical section) waits for lock 2 that is owned by thread B, while thread B is waiting for lock 1.

The way to avoid deadlocks is theoretically easy: always acquire the locks in the same order. This means that every thread that needs more than one lock should always acquire the locks in the same order. This guarantees deadlock cannot happen (at least not because of these locks). The practical issue is how to enforce the ordering; without writing any code, it's a matter of documenting the order so that future code continues to adhere to the rules. An alternative option is to write a “multi-lock” wrapper that always acquires locks in the same order. A simple way to accomplish that is to order the acquisition by the lock's address in memory.



Write such a multi-lock wrapper for critical sections.

The MD5 Calculator Application

The *MD5Calculator* application demonstrates the use of critical sections that is more interesting (and complex) than *Simple Increment* (we'll also modify it in later sections). The

application calculates the MD5 hash of image files (EXEs and DLLs) as they are loaded by processes. Since processes typically use many common DLLs, it's better to cache the results of hashes that have already been calculated. The application has several challenges:

- Show a responsive user interface while calculations are done in the background.
- Get notified of new images (DLLs/EXEs) that are loaded by any process on the system.
- Manage a cache of files with their MD5 hashes.

Figure 7-7 shows the main screen of the application before any activity.

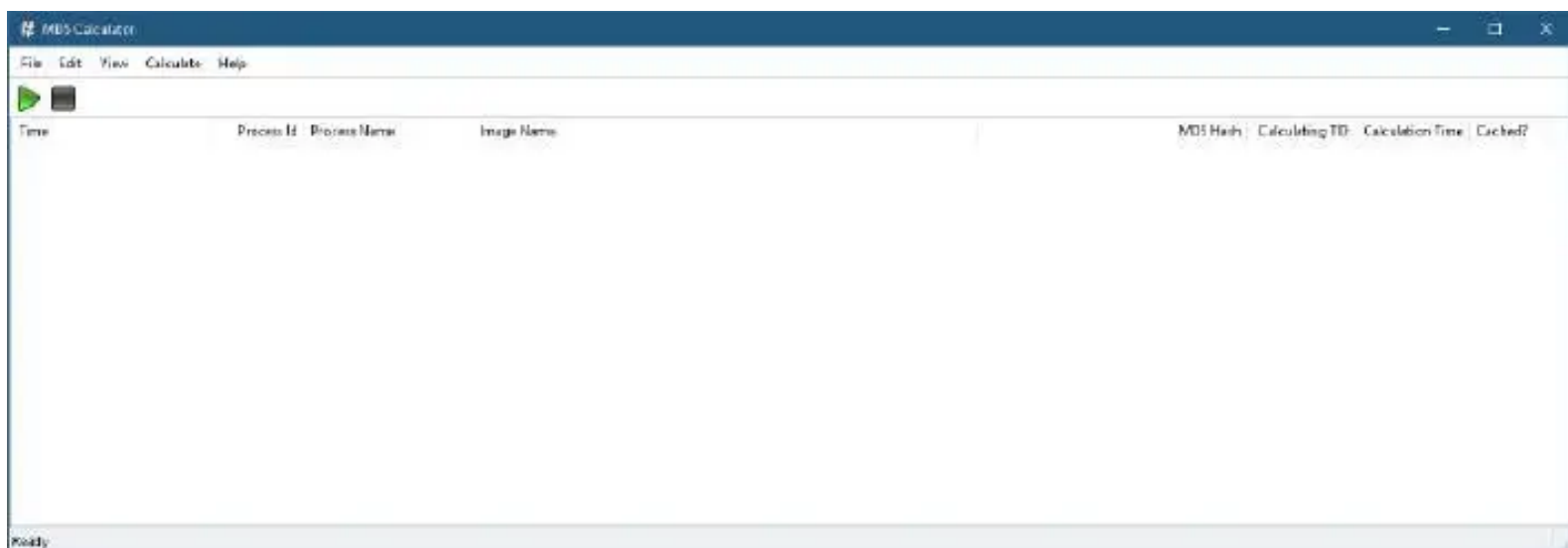


Figure 7-7: The *MD5 Calculator* application

By default, no caching for MD5 hashes is done. Clicking the green *Go* button (or the *Calculate* / *Go* menu item), starts capturing image loads and calculates hashes from scratch for each image file, even if duplicates are loaded (figure 7-8).

Time	Process ID	Process Name	Image Name	MD5 Hash	Calculating T...	Calculation T...	Cached?
02/14/2019 4:25:46.665	19304	SnagitEditor.exe	C:\Windows\System32\OcsDemandLoadResourceHelper.dll	9D1E318E09783A1B8F5F10619950662	21780	1480 urex	No
02/14/2019 4:25:47.065	19304	SnagitEditor.exe	C:\Windows\System32\FWPUCNT.DLL	8D64320C630493864991770C9399804	29100	1656 urex	No
02/14/2019 4:25:48.1774	40572		C:\Windows\Microsoft.NET\Framework\v4.0.30319\clbcatq.dll	68C1A126851D7D13ED1AD7D5E93D2E0	27640	552 urex	No
02/14/2019 4:25:48.566	40572		C:\Windows\System32\W6atmarta.dll	CF4C18621BF9D11E75CD76D4CFDA7A1	26980	498 urex	No
02/14/2019 4:25:48.120201	41580		C:\Windows\System32\W6atmarta.dll	CF4C18621BF9D11E75CD76D4CFDA7A1	17000	492 urex	No
02/14/2019 4:25:48.132344	41580		C:\Windows\System32\W6atmarta.dll	CF4C18621BF9D11E75CD76D4CFDA7A1	38548	512 urex	No
02/14/2019 4:25:48.156484	34292		C:\Program Files (x86)\Microsoft Visual Studio\2019\Preview\VC\Tools\MSVC\14.2...	00DC206CD47C71841064DDA1D1DC08F9	20002	742 urex	No
02/14/2019 4:25:48.156467	34292		C:\Windows\System32\ntdll.dll	A82E997718297B9C7A880E1F1D00D23	38784	4701 urex	No
02/14/2019 4:25:48.156468	34292		C:\Windows\System32\W6atmarta.dll	CF4C18621BF9D11E75CD76D4CFDA7A1	1692	3924 urex	No
02/14/2019 4:25:48.156505	34292		C:\Windows\System32\wsm64.dll	F18CA316570C115F8E581E2C807F	14448	852 urex	No
02/14/2019 4:25:48.156510	34292		C:\Windows\System32\wsm64win.dll	9F1A3754C54C67C663908CF58666BC	31924	1216 urex	No
02/14/2019 4:25:48.156539	34292		C:\Windows\System32\wsm64cpu.dll	7FF428F98A32EBC873D7A16187F0A89	25716	228 urex	No
02/14/2019 4:25:48.156587	34292		C:\Windows\System32\W6atmarta.dll	9994FED79F14718D9487648C3081C7	37796	1053 urex	No
02/14/2019 4:25:48.156592	34292		C:\Windows\System32\W6atmarta.dll	0091518D5706A8D70D1163F888208E	11980	1792 urex	No
02/14/2019 4:25:48.156598	34292		C:\Windows\System32\W6atmarta.dll	34EBAE24D8FCD8977AC23848D457F62	34844	5084 urex	No
02/14/2019 4:25:48.156716	34292		C:\Windows\System32\W6atmarta.dll	3D0C21D5C03D0E513D04C0D640D0147	26296	1941 urex	No
02/14/2019 4:25:48.156849	34292		C:\Windows\System32\W6atmarta.dll	ACF08D91AC5D56A14C38B5C771B2D4	11208	1644 urex	No
02/14/2019 4:25:48.156857	34292		C:\Windows\System32\W6atmarta.dll	14C80D032CDD56100C5385CDCA86A6	21080	1001 urex	No
02/14/2019 4:25:48.156864	34292		C:\Windows\System32\W6atmarta.dll	5968D9D03A1F1917CE9968E31F529D	26156	1078 urex	No
02/14/2019 4:25:48.156875	34292		C:\Program Files (x86)\Microsoft Visual Studio\2019\Preview\VC\Tools\MSVC\14.2...	4C36F78D61F56AA5F11065FAC54B4	17504	343 urex	No
02/14/2019 4:25:48.156876	34292		C:\Windows\System32\W6atmarta.dll	C770B77D0C21D4708C52614C52614F	41784	1656 urex	No
02/14/2019 4:25:48.156886	34292		C:\Windows\System32\W6atmarta.dll	59C025C804C101B0F706471AD820A82	33448	2470 urex	No
02/14/2019 4:25:48.156895	34292		C:\Program Files (x86)\Microsoft Visual Studio\2019\Preview\VC\Tools\MSVC\14.2...	49732025E5C1811361E15161007086	21108	1088 urex	No
02/14/2019 4:25:48.156919	34292		C:\Windows\System32\W6atmarta.dll	E10B8E37F0481A5B7716188C62D495C	39616	405 urex	No
02/14/2019 4:25:48.156922	34292		C:\Windows\System32\W6atmarta.dll	4668D71AC3DC18505A5568E6A3C21C2	16176	284 urex	No
02/14/2019 4:25:48.156939	34292		C:\Windows\System32\W6atmarta.dll	9C7A21F81375D59C08F59794B0B5EC	2084	517 urex	No
02/14/2019 4:25:48.156940	34292		C:\Windows\System32\W6atmarta.dll	4C80C404670100D5D3C78D015A4D	21020	2180 urex	No
02/14/2019 4:25:48.156963	34292		C:\Windows\System32\W6atmarta.dll	91A379A2E5C71F588E4C14091AD1D08	38880	5251 urex	No
02/14/2019 4:25:48.156976	34292		C:\Windows\System32\W6atmarta.dll	4C7FB3E3A5A704644D7C33C42B87651	30664	409 urex	No
02/14/2019 4:25:48.156986	34292		C:\Windows\System32\W6atmarta.dll	C6F8EC7A6E3129CFD8358387F623E3	41388	357 urex	No

Figure 7-8: The MD5 Calculator application running with no caching

You can launch a new process, such as *Notepad* and watch its modules load and reflected at the bottom of the list view display. Click the *Stop* button to stop capturing image loads. You can use the *Edit / Clear* menu item to clear the display.

Now you can use the *Calculate / Use Cache* menu item to toggle cache usage. Now click *Go* again. Notice that after some calculations, the cache starts to be useful and the “Cached?” column shows more “Yes” items when the hash value can be satisfied by the cache (figure 7-9).

Time	Process ID	Process Name	Image Name	MD5 Hash	Calculating T...	Calculation T...	Cached?
02/14/2019 5:25:40.745	41704	notepad.exe	C:\Windows\System32\notepad.dll	7C9D034E D633 F131 8A B C 41 20 6 F F C 9 E	18066	436 usec	No
02/14/2019 5:25:40.841	41704	notepad.exe	C:\Windows\System32\winapiutils.dll	3D1B7939540E B288556CAF7916E966	25624	5994 usec	No
02/14/2019 5:25:40.862	41704	notepad.exe	C:\Windows\System32\ole32.dll	F4F94B4F B42C 4E1F 4B4C C6B010 F7E6	31604	731 usec	No
02/14/2019 5:25:40.886	41704	notepad.exe	C:\Windows\System32\ole32.dll	9C4096FC DC877E1F30B325A47555ADE	13968	1633 usec	No
02/14/2019 5:25:40.891	41704	notepad.exe	C:\Windows\System32\ole32.dll	3A9664D73A F3B11406C C579DDE52740	26012	1192 usec	No
02/14/2019 5:25:40.926	41704	notepad.exe	C:\Windows\System32\ole32.dll	A0E1808797090A 1A425B1B713917F5B	31736	445 usec	No
02/14/2019 5:25:40.921	41704	notepad.exe	C:\Windows\System32\ole32.dll	A0B0A11C1A7A24FC044070A D2FD8064	27100	1084 usec	No
02/14/2019 5:25:40.758	41704	notepad.exe	C:\Windows\System32\ole32.dll	65CF1A610C5D2E7184D879196755C38E	35432	1966 usec	No
02/14/2019 5:25:40.716	41704	notepad.exe	C:\Windows\System32\ole32.dll	FC70448827762446A408304480BA8C84	26864	7436 usec	No
02/14/2019 5:25:40.718	41704	notepad.exe	C:\Windows\System32\ole32.dll	64C146E1980001144A1D279071C96A	34696	2063 usec	No
02/14/2019 5:25:40.739	41704	notepad.exe	C:\Windows\System32\ole32.dll	4360901D0C3658FA34B9C803F85286	20716	535 usec	No
02/14/2019 5:25:40.802	41704	notepad.exe	C:\Windows\System32\ole32.dll	5800E203DC34E8A2B4683A74C6A06E	39040	6257 usec	No
02/14/2019 5:31:11.2348	11892	explorer.exe	C:\Program Files\Common-Files\Microsoft\Shared\ink\inkapi.dll	B8716C6DC57F2345CC41307238F480B9			Yes
02/14/2019 5:31:18.9418	15116		C:\Windows\System32\ole32.dll	R9BCD04134C559D4E1C1B1D678162A8A	20240	202 usec	No
02/14/2019 5:31:18.9530	15116		C:\Windows\System32\ole32.dll	A8E397718207B9C7A880E1A0D00033			Yes
02/14/2019 5:31:18.962	15116		C:\Windows\System32\ole32.dll	0F28D0D1DC506F561C1C48F4F5D0DC			Yes
02/14/2019 5:31:18.969	15116		C:\Windows\System32\ole32.dll	982624FF18080B93C4C1912457526F			Yes
02/14/2019 5:31:18.970	15116		C:\Windows\System32\ole32.dll	6C4096FC DC877E1F30B325A47555ADE			Yes
02/14/2019 5:31:18.984	15116		C:\Windows\System32\ole32.dll	81199C7BC06AC3805A8B9042124A547			Yes
02/14/2019 5:31:18.992	15116		C:\Windows\System32\ole32.dll	3A9544073A F3B11406C C579DDE52740			Yes
02/14/2019 5:31:18.994	15116		C:\Windows\System32\ole32.dll	84FD4A29E5A69F8887AC1A7B14C1C58			Yes
02/14/2019 5:31:18.996	15116		C:\Windows\System32\ole32.dll	30154D8B58A54A4B9419101852C0A42			Yes
02/14/2019 5:31:18.997	15116		C:\Windows\System32\ole32.dll	000063C4011D4D2090CCE1E36586D802			Yes
02/14/2019 5:31:18.997	15116		C:\Windows\System32\ole32.dll	483AA25B8F D213899AC2D1168913			Yes
02/14/2019 5:31:18.997	15116		C:\Windows\System32\ole32.dll	13E0DF15001519E0C0A014C4DC042C			Yes
02/14/2019 5:31:18.998	15116		C:\Windows\System32\ole32.dll	54B72D3C3E8B1D3D18A3A000B8F47			Yes
02/14/2019 5:31:18.999	15116		C:\Windows\System32\ole32.dll	AACA1005705134D913A7FDC70C0C02			Yes
02/14/2019 5:31:18.999	15116		C:\Windows\System32\ole32.dll	E0D38E74453A1E1E1D72AC1515B8457			Yes
02/14/2019 5:31:18.999	15116		C:\Windows\System32\ole32.dll	C90B4C1D60F56657CA11C81A5E6B3A3F			Yes
02/14/2019 5:31:18.999	15116		C:\Windows\System32\ole32.dll	8D585804DC75C5A3A09C9C4A4C9C1C7			Yes

Figure 7-9: The MD5 Calculator application running with caching

Let's walk through the most important parts of the application.

Calculating MD5 Hash

Calculating the MD5 hash of any buffer can be done using the Windows cryptographic API. A simple class named MD5Calculator is used to make the calculation (part of its own static library project named *HashCalc*):

```
// MD5Calculator.h
```

```
class MD5Calculator {
public:
    static std::vector<uint8_t> Calculate(PCWSTR path);
};
```

```
// MD5Calculator.cpp
```

```
#include <bcrypt.h>
#include "MD5Calculator.h"
#include <wil\resource.h>
```

```

std::vector<uint8_t> MD5Calculator::Calculate(PCWSTR path) {
    std::vector<uint8_t> md5;

    wil::unique_hfile hFile(::CreateFile(path, GENERIC_READ, FILE_SHARE_READ,
        nullptr, OPEN_EXISTING, FILE_FLAG_SEQUENTIAL_SCAN, nullptr));
    if (!hFile)
        return md5;

    wil::unique_handle hMemMap(::CreateFileMapping(hFile.get(), nullptr,
        PAGE_READONLY, 0, 0, nullptr));
    if (!hMemMap)
        return md5;

    wil::unique_hcryptprov hProvider;
    if (!::CryptAcquireContext(hProvider.addressof(), nullptr, nullptr,
        PROV_RSA_FULL, CRYPT_VERIFYCONTEXT))
        return md5;

    wil::unique_hcrypthash hHash;
    if (!::CryptCreateHash(hProvider.get(), CALG_MD5, 0, 0, hHash.addressof()))
        return md5;

    wil::unique_mapview_ptr<BYTE> buffer((BYTE*)::MapViewOfFile(hMemMap.get(),
        FILE_MAP_READ, 0, 0, 0));
    if (!buffer)
        return md5;

    auto size = ::GetFileSize(hFile.get(), nullptr);
    if (!::CryptHashData(hHash.get(), buffer.get(), size, 0))
        return md5;

    DWORD hashSize;
    DWORD len = sizeof(DWORD);
    if (!::CryptGetHashParam(hHash.get(), HP_HASHSIZE, (BYTE*)&hashSize,
        &len, 0))
        return md5;

    md5.resize(len = hashSize);
    ::CryptGetHashParam(hHash.get(), HP_HASHVAL, md5.data(), &len, 0);
}

```

```

    return md5;
}

```

A detailed description of the cryptographic API is beyond the scope of this book. However, the procedure to calculate a hash is fairly straightforward. There are a few preparatory steps, before calling the actual hashing function, `CryptHashData`. This function accepts a handle representing the hashing algorithm, the buffer to hash and the size of the buffer to hash.

Here are the steps, in order, performed by `MD5Calculator::Calculate`:

1. The file in question is opened with `CreateFile` (discussed in detail in chapter 11). The file is opened for read access, with the optional flag `FILE_FLAG_SEQUENTIAL_SCAN` that is a hint to the file system that the reading is going to be sequential.
2. The contents of the file must be placed in a memory buffer to be usable by the hashing function. One way would be to allocate a buffer the size of the file and use `ReadFile` to read its contents into the buffer. A better approach is to use a *memory-mapped file* (discussed in chapter 14), that can map a file's contents to memory (see item 5 below), without any need to allocate or read anything. The `CreateFileMapping` function is used to create the file mapping object based on the file handle (first argument).
3. `CryptAcquireContext` is called to get back a cryptographic provider handle based a provider (`PROV_RSA_FULL` in our case).
4. The call to `CryptCreateHash` returns a handle to a specific hash algorithm (`CALG_MD5` for MD5).
5. Calling `MapViewOfFile` maps the file contents to memory, returning a pointer. This is wrapped by the WIL `unique_mapview_ptr<>` that calls `UnmapViewOfFile` when the variable goes out of scope.
6. Now everything is ready to call `CryptHashData` to calculate the hash.
7. All that's left to do is retrieve the hash size and hash data itself. Both are accomplished with calls to `CryptGetHashParam`: the first with `HP_HASHSIZE` to get the hash size (always 16 bytes for MD5, but the code remains generic).
8. The buffer for the result is allocated by calling `resize` on the vector of bytes. Then a second call to `CryptGetHashParam` is made with `HP_HASHVAL` to get the actual hash.

The Hash Cache

The cache itself is encapsulated in the `HashCache` class defined like so:

```

using Hash = std::vector<uint8_t>;

class HashCache {
public:
    HashCache();

    bool Add(PCWSTR path, const Hash& hash);
    const Hash Get(PCWSTR path) const;
    bool Remove(PCWSTR path);
    void Clear();

private:
    mutable CriticalSection _lock;
    std::unordered_map<std::wstring, Hash> _cache;
};

```

The cache is managed with an `unordered_map<>` object from the C++ standard library that maps a file path to its hash. The hash itself is stored as a vector of bytes, although for MD5 I could have just used an array of 16 bytes. Since accessing the cache may be done by multiple threads, the `unordered_map<>` must be protected from concurrent access. Here I'm using a critical section. The implementation is fairly straightforward, protecting each operation with the critical section:

```

HashCache::HashCache() {
    _cache.reserve(512);
}

bool HashCache::Add(PCWSTR path, const Hash& hash) {
    AutoCriticalSection locker(_lock);
    auto it = _cache.find(path);
    if (it == _cache.end()) {
        _cache.insert({ path, hash });
        return true;
    }
    return false;
}

const Hash HashCache::Get(PCWSTR path) const {
    AutoCriticalSection locker(_lock);

```

```

    auto it = _cache.find(path);
    return it == _cache.end() ? Hash() : it->second;
}

bool HashCache::Remove(PCWSTR path) {
    AutoCriticalSection locker(_lock);
    auto it = _cache.find(path);
    if (it != _cache.end()) {
        _cache.erase(it);
        return true;
    }
    return false;
}

void HashCache::Clear() {
    AutoCriticalSection locker(_lock);
    _cache.clear();
}

```

The code uses the RAII `AutoCriticalSection` class, defined earlier to acquire and release with no need for termination handlers (`__try` / `__finally`).

The main view class (`CView`) holds an instance of `HashCache` named `m_Cache`, that is used if cache usage is enabled (`m_UseCache` member).

Image Loads Notifications

The next, relatively independent, piece of the puzzle is getting notifications about image loads. One powerful way to get these notifications from user mode is to utilize *Event Tracing for Windows* (ETW). ETW is a mechanism existed since Windows 2000 that allows system components and other applications to generate rich events that can be consumed in real-time or logged to a file and analyzed later. A basic ETW architecture is shown in figure 7-10.

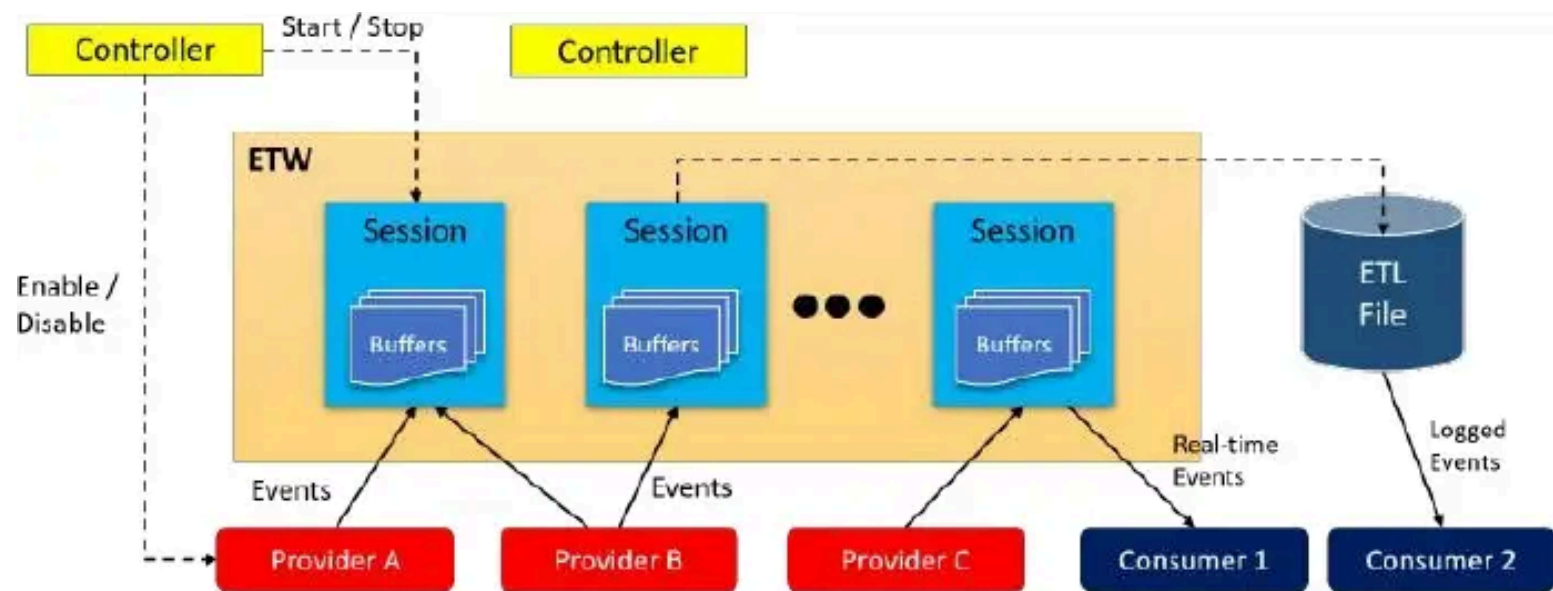


Figure 7-10: ETW Architecture

The main pieces of ETW are the following:

- Providers generate events.
- Sessions encapsulate one or more providers plus some configuration. Events are captured when the session starts until it's stopped.
- Controllers enable or disable providers, and start and stop sessions.
- Consumers consume events in real-time or to a file (*ETL* - event tracing log). In a typical case, a controller is also a consumer.

Complete treatment of ETW is beyond the scope of this book.

In our case, we need to use the kernel provider that can send a bunch of events, one of them is image loads. The `TraceManager` class encapsulates working with the ETW infrastructure. It's defined like so:

```

class TraceManager final {
public:
    ~TraceManager();

    bool Start(std::function<void(PEVENT_RECORD)> callback);
    bool Stop();

private:
    void OnEventRecord(PEVENT_RECORD rec);
    DWORD Run();

private:
    TRACEHANDLE _handle{ 0 };
    TRACEHANDLE _hTrace{ 0 };
    EVENT_TRACE_PROPERTIES* _properties;
    std::unique_ptr<BYTE[]> _propertiesBuffer;
    EVENT_TRACE_LOGFILE _traceLog = { 0 };
    wil::unique_handle _hProcessThread;
    std::function<void(PEVENT_RECORD)> _callback;
};

```

The interface exposed by `TraceManager` is fairly simple. Once constructed, a session is started by calling `Start` and stopped by calling `Stop`. The private `Run` method is the one that starts the session running with its own thread (more on that in a moment). The `OnEventRecord` function is a callback invoked for every generated event. The various private data members are mostly concerned with building an ETW session and managing it. Let's look at the implementation.

The destructor just calls `Stop`:

```

TraceManager::~TraceManager() {
    Stop();
}

```

`Start` is a bulky function that sets up the ETW session appropriately and then initiates processing. It accepts a callback from an interested consumer to be called for each event. Its first major task is calling `StartTrace`, that configures and starts a session:

```

bool TraceManager::Start(std::function<void(PEVENT_RECORD)> cb) {
    _callback = cb;

    if (_handle || _hTrace)
        return true;

    auto size = sizeof(EVENT_TRACE_PROPERTIES) + sizeof(KERNEL_LOGGER_NAME);
    _propertiesBuffer = std::make_unique<BYTE[]>(size);
    ::memset(_propertiesBuffer.get(), 0, size);

    _properties = reinterpret_cast<EVENT_TRACE_PROPERTIES*>(_propertiesBuffer.get());
    _properties->EnableFlags = EVENT_TRACE_FLAG_IMAGE_LOAD;
    _properties->Wnode.BufferSize = (ULONG)size;
    _properties->Wnode.Guid = SystemTraceControlGuid;
    _properties->Wnode.Flags = WNODE_FLAG_TRACED_GUID;
    _properties->Wnode.ClientContext = 1;
    _properties->LogFileMode = EVENT_TRACE_REAL_TIME_MODE;
    _properties->LoggerNameOffset = sizeof(EVENT_TRACE_PROPERTIES);

    auto error = ::StartTrace(&_amp;_handle, KERNEL_LOGGER_NAME, _properties);
    if (error != ERROR_SUCCESS && error != ERROR_ALREADY_EXISTS)
        return false;
}

```

First, it checks if a session is already in progress, and if so, it simply returns. Otherwise, it saves the callback in a data member and goes on to prepare the `EVENT_TRACE_PROPERTIES` structure. The important parts are setting `EnableFlags` to `EVENT_TRACE_FLAG_IMAGE_LOAD`, which specifies that image-related events are of interest, and `LogFileMode` to `EVENT_TRACE_REAL_TIME_MODE` to indicate a real-time session is requested.

Check the documentation of `StartTrace` for the full details.

ETW sessions are unique in the sense that they can outlive a process. This means `StartTrace` can fail, but if the last error is `ERROR_ALREADY_EXISTS` then the session is already running, and we can just tap into it as a consumer.

Next, we need to set up the consumer, by calling `OpenTrace`:

```

_traceLog.Context = this;
_traceLog.LoggerName = KERNEL_LOGGER_NAME;
_traceLog.ProcessTraceMode = PROCESS_TRACE_MODE_EVENT_RECORD |
    PROCESS_TRACE_MODE_REAL_TIME;
_traceLog.EventRecordCallback = [](PEVENT_RECORD record) {
    ((TraceManager*)record->UserContext)->OnEventRecord(record);
};
_hTrace = ::OpenTrace(&_traceLog);
if (!_hTrace)
    return false;

```

The callback for each event is set in the `EventRecordCallback` member of the `EVENT_TRACE_LOGFILE` structure (`_traceLog`). It uses the `UserContext` member as the `this` pointer (set earlier in the `Context` member) to invoke an instance function of the class. This function (`OnEventRecord`) will invoke the callback that was passed in earlier in `Start`.

The consumer is now in place, so the last operation needed is to start processing events. To that end, a separate thread is created, because the `ProcessTrace` function is a blocking one, and we don't want the caller to be blocked when calling `Start`:

```

_hProcessThread.reset(::CreateThread(nullptr, 0, [](auto param) {
    return ((TraceManager*)param)->Run();
}, this, 0, nullptr));

return true;
}

```

The `Run` member function just calls `ProcessTrace`:

```

DWORD TraceManager::Run() {
    auto error = ::ProcessTrace(&_hTrace, 1, nullptr, nullptr);
    return error;
}

```

As mentioned, the `OnEventRecord` function invokes the client's callback:

```

void TraceManager::OnEventRecord(PEVENT_RECORD rec) {
    if (_callback)
        _callback(rec);
}

```

Finally, the Stop function closes and stops the trace:

```

bool TraceManager::Stop() {
    if (_hTrace) {
        ::CloseTrace(_hTrace);
        _hTrace = 0;
    }
    if (_handle) {
        ::StopTrace(_handle, KERNEL_LOGGER_NAME, _properties);
        _handle = 0;
    }
    return true;
}

```

The main frame class (CMainFrame) holds a TraceManager instance. It calls Start and Stop when the appropriate menu / tool bar items are selected:

```

LRESULT CMainFrame::OnStartTrace(WORD, WORD, HWND, BOOL&) {
    m_TraceManager.Start([this](auto record) {
        m_view.OnEvent(record);    // call the view
    });

    // UI updates omitted...

    return 0;
}

LRESULT CMainFrame::OnStopTrace(WORD, WORD, HWND, BOOL&) {
    m_TraceManager.Stop();

    // UI updates omitted

    return 0;
}

```

Event Parsing

An ETW event is provided to the event callback using the `EVENT_RECORD` structure that contains everything about the specific event. Here is its definition:

```
typedef struct _EVENT_RECORD {
    EVENT_HEADER      EventHeader;           // Event header
    ETW_BUFFER_CONTEXT BufferContext;        // Buffer context
    USHORT            ExtendedDataCount;     // Number of extended
                                           // data items
    USHORT            UserDataLength;        // User data length
    PEVENT_HEADER_EXTENDED_DATA_ITEM ExtendedData; // Pointer to an array of
                                           // extended data items
    PVOID             UserData;              // Pointer to user data
    PVOID             UserContext;          // Context from OpenTrace
} EVENT_RECORD, *PEVENT_RECORD;
```

ETW event properties can include strings (ANSI and Unicode), numbers, custom structures, and some other special types. Everything is stored in a binary blob as part of `EVENT_RECORD` starting at `UserData` address. To get to the various properties and values, some parsing is required. The `EventParser` class is a helper class for parsing properties. It stores each parsed property in an `EventProperty` structure defined like so:

```
struct EventProperty {
    EventProperty(EVENT_PROPERTY_INFO& info);

    std::wstring Name;
    BYTE* Data;
    ULONG Length;
    EVENT_PROPERTY_INFO& Info;

    template<typename T>
    T GetValue() const {
        static_assert(std::is_pod<T>() && !std::is_pointer<T>());
        return *(T*)Data;
    }

    PCWSTR GetUnicodeString() const;
```

```
PCSTR GetAnsiString() const;
};
```

Each property has a name (Name member), a pointer to the actual data (Data), a length (Length) and a reference to the original structure describing the property (Info). The GetValue<> template function retrieves property values for simple POD (“plain old data”) types, such as numeric types. The static_assert statement instructs the compiler to reject complex types, as they would produce a wrong value.

static_assert was introduced in C++ 11 and enhanced in C++ 14.

GetUnicodeString and GetAnsiString return the data as their corresponding string types. The EventParser class is declared like so:

```
class EventParser {
public:
    EventParser(PEVENT_RECORD record);

    PTRACE_EVENT_INFO GetEventInfo() const;
    PEVENT_RECORD GetEventRecord() const;
    const EVENT_HEADER& GetEventHeader() const;
    const std::vector<EventProperty>& GetProperties() const;
    const EventProperty* GetProperty(PCWSTR name) const;    // lookup by name

    DWORD GetProcessId() const;

    static std::wstring GetDosNameFromNtName(PCWSTR name);

private:
    std::unique_ptr<BYTE[]> _buffer;
    PTRACE_EVENT_INFO _info{ nullptr };
    PEVENT_RECORD _record;
    mutable std::vector<EventProperty> _properties;
};
```

An EventParser instance takes an EVENT_RECORD as its input. Everything about the event

is stored there somewhere. EventParser's task is to extract the required information. Here is the implementation except for GetDosNameFromNtName, which we'll deal with separately:

```

EventProperty::EventProperty(EVENT_PROPERTY_INFO& info) : Info(info) {
}

EventParser::EventParser(PEVENT_RECORD record) : _record(record) {
    ULONG size = 0;
    auto error = ::TdhGetEventInformation(record, 0, nullptr, _info, &size);
    if (error == ERROR_INSUFFICIENT_BUFFER) {
        _buffer = std::make_unique<BYTE[]>(size);
        _info = reinterpret_cast<PTRACE_EVENT_INFO>(_buffer.get());
        error = ::TdhGetEventInformation(record, 0, nullptr, _info, &size);
    }
    ::SetLastError(error);
}

PTRACE_EVENT_INFO EventParser::GetEventInfo() const {
    return _info;
}

PEVENT_RECORD EventParser::GetEventRecord() const {
    return _record;
}

const EVENT_HEADER& EventParser::GetEventHeader() const {
    return _record->EventHeader;
}

const std::vector<EventProperty>& EventParser::GetProperties() const {
    if (!_properties.empty())
        return _properties;

    _properties.reserve(_info->TopLevelPropertyCount);
    auto userDataLength = _record->UserDataLength;
    BYTE* data = (BYTE*)_record->UserData;

    for (ULONG i = 0; i < _info->TopLevelPropertyCount; i++) {
        auto& prop = _info->EventPropertyInfoArray[i];
    }
}

```

```

        EventProperty property(prop);
        property.Name.assign((WCHAR*)((BYTE*)_info + prop.NameOffset));
        auto len = prop.length;
        property.Length = len;
        property.Data = data;
        data += len;
        userDataLength -= len;

        _properties.push_back(std::move(property));
    }

    return _properties;
}

const EventProperty* EventParser::GetProperty(PCWSTR name) const {
    for (auto& prop : GetProperties())
        if (prop.Name == name)
            return &prop;
    return nullptr;
}

DWORD EventParser::GetProcessId() const {
    return _record->EventHeader.ProcessId;
}

PCWSTR EventProperty::GetUnicodeString() const {
    return (PCWSTR)Data;
}

PCSTR EventProperty::GetAnsiString() const {
    return (PCSTR)Data;
}

```

The constructor calls `TdhGetEventInformation` to get the basic event details and an array of properties, all from the `EVENT_RECORD`. The call is made twice: the first time with a length of zero to get the required length, then, after allocating the required buffer, makes a second call to retrieve the actual data.



The Tdh functions require the header `<tdh.h>` and linking against `tdh.lib`. As mentioned earlier, a detailed discussion of these functions is outside the scope of this book.

`GetProperties` does the hard work of walking each property in the event, extracting the important information and encapsulating it in an `EventProperty` instance. The `GetProperty` helper function returns a property given its name (if any).

Putting it All Together

Now that we have all the individual pieces, we can start integrating them into an actual application. The main view class (`CView`) holds the data items that are actually displayed. These take the form of `EventData` structures defined like so (in `view.h`):

```
struct EventData {
    CString FileName;
    ULONGLONG Time;
    DWORD ProcessId;
    Hash MD5Hash;
    DWORD CalculatingThreadId;
    DWORD CalculationTime;
    bool Cached : 1;
    bool CalcDone : 1;
};
```

The view stores a vector of these items, a critical section to protect access to the vector, the cache itself, and whether it should be used:

```
class CView... {
//...
private:
    std::vector<EventData> m_Events;
    HashCache m_Cache;
    CriticalSection m_EventsLock;
    bool m_UseCache{ false };
};
```

When an event comes in, the callback `OnEvent` is invoked. The callback needs to grab the event details, store them in a new `EventData` object and go ahead and calculate the MD5 hash (or use a cached result). First, it filters out unwanted events:

```
void CView::OnEvent(PEVENT_RECORD record) {
    EventParser parser(record);
    // ID 10 is a load image event
    if (parser.GetEventHeader().EventDescriptor.Opcode != 10)
        return;
```

Remember the ETW trace only enables the image load type of events. As it turns out, there are actually four events that may be sent. Only one of them (with an opcode of 10) is an image load. Refer to [this reference](#)³ for the full details.

Here is the structure for the Image load set of events:

```
[EventType(10, 2, 3, 4), EventType("Load", "Unload", "DCStart", "DCEnd")]
class Image_Load : Image {
    uint32 ImageBase;
    uint32 ImageSize;
    uint32 ProcessId;
    uint32 ImageChecksum;
    uint32 TimeDateStamp;
    uint32 Reserved0;
    uint32 DefaultBase;
    uint32 Reserved1;
    uint32 Reserved2;
    uint32 Reserved3;
    uint32 Reserved4;
    string FileName;
};
```

The above format is known as *simplified MOF* (Managed Object Format). It's used with ETW and *Windows Management Instrumentation* (WMI).

³<https://docs.microsoft.com/en-us/windows/win32/etw/image-load>

Once the proper event is received (Opcode=10), the interesting property (*FileName*) is retrieved and used to fill a new *EventData* instance, before initiating computation of the MD5 hash by posting a custom message to the view:

```

auto fileName = parser.GetProperty(L"FileName");
if (fileName) {
    EventData data;
    data.FileName = parser.GetDosNameFromNtName(
        fileName->GetUnicodeString()).c_str();
    data.ProcessId = parser.GetProcessId();
    data.Time = parser.GetEventHeader().TimeStamp.QuadPart;
    data.CalcDone = false;
    size_t size;
    {
        AutoCriticalSection locker(m_EventsLock);
        m_Events.push_back(std::move(data));
        size = m_Events.size();
    }
    int index = static_cast<int>(size - 1);

    // initiate work from the UI thread

    PostMessage(WM_START_CALC, index, size);
}
}

```

There are a few points worth noting:

- The critical section protecting the view's data is held for as short time as possible. This is achieved by using an artificial block so that the critical section can be released as soon as the block is exited.
- Initiating the calculation is not done in this function, as it's called by the *TraceManager*'s thread, which should be let go as soon as possible to be available to process the next event. Instead, calling *PostMessage* causes a message to be asynchronously sent to the window, to be handled by the UI thread, allowing the current function to return.

The last interesting detail of the above code is the use *GetDosNameFromNtName*. The file name provided by the ETW event is in “device format”, which is the native NT

format looking something like this: “DeviceHarddiskVolume3SomeDirectorySomeFile.dll”. The reason internal device names have this format will be discussed in chapter 11. For now, this kind of path should be translated to something like “c:SomeDirectorySomeFile.dll” to be usable by the `CreateFile` API used for the hash calculation. The static `EventParser::GetDosNameFromNtName` is used to translate back to a Win32 device name:

```
std::wstring EventParser::GetDosNameFromNtName(PCWSTR name) {
    static std::vector<std::pair<std::wstring, std::wstring>> deviceNames;
    static bool first = true;

    if (first) {
        auto drives = ::GetLogicalDrives();
        int drive = 0;
        while (drives) {
            if (drives & 1) {
                // drive exists
                WCHAR driveName[] = L"X:";
                driveName[0] = (WCHAR)(drive + 'A');
                WCHAR path[MAX_PATH];
                if (::QueryDosDevice(driveName, path, MAX_PATH)) {
                    deviceNames.push_back({ path, driveName });
                }
            }
            drive++;
            drives >>= 1;
        }
        first = false;
    }

    for (auto& [ntName, dosName] : deviceNames) {
        if (::wcsnicmp(name, ntName.c_str(), ntName.size()) == 0)
            return dosName + (name + ntName.size());
    }
    return L"";
}
```

The function uses a static vector of string pairs, each mapping an NT device name to a drive letter. The first time the function is called, it gets all the driver letters (`GetLogicalDrives`) as a bitmask, where bit 0 corresponds to drive A, bit 1 to drive B, bit 2 to driver C, and so on,

and for each drive queries for its NT device name with `QueryDosDevice` (see chapter 11 for more on `QueryDosDevice`).

With the passed-in path at hand, the device name is searched in the vector and its corresponding driver letter is extracted. Finally, the rest of the path is appended to the extracted drive letter and returned to the caller.

Reader Writer Locks

Using critical sections to protect shared data from concurrent access works well, but it's a pessimistic mechanism - it allows one thread at most to access the shared data. In some scenarios, where some threads read the data, and other threads write the data, an optimization can be made: If one thread reads the data, there is no reason to prevent other threads that only read the data from doing it concurrently. This is exactly the role of the "Single Writer Multiple Readers" mechanism.

The Windows API provides the `SRWLOCK` structure that represents such a lock (S is for "Slim"). its definition is as follows:

```
typedef RTL_SRWLOCK SRWLOCK, *PSRWLOCK;
```

What about that `RTL_SRWLOCK`? Here goes:

```
typedef struct _RTL_SRWLOCK {  
    PVOID Ptr;  
} RTL_SRWLOCK, *PRTL_SRWLOCK;
```

Clearly, this is just an opaque data that should be treated as such.

Initializing an `SRWLOCK` is accomplished with `InitializeSRWLock`:

```
void InitializeSRWLock(_Out_ PSRWLOCK SRWLock);
```

Alternatively, the structure can be initialized statically by assigning it to `SRWLOCK_INIT` macro, which just zeros out the structure. Curiously enough, there is no "delete" for the `SRWLOCK`; this is because all of its internal information is packed into that pointer-sized cell.

With an initialized `SRWLOCK`, threads can attempt to acquire the shared or exclusive lock using the following functions:

```
void AcquireSRWLockShared    (_Inout_ PSRWLOCK SRWLock);  
void AcquireSRWLockExclusive (_Inout_ PSRWLOCK SRWLock);
```

If the relevant lock cannot be acquired, the thread enters a wait state. Once acquired, the thread can make forward progress and access the shared resource as specified, meaning it's up to the thread not to do the “wrong” access. For example, if a thread acquires the shared lock, it must not modify the shared data.

Once the work is done, the thread uses the associated release function:

```
void ReleaseSRWLockShared    (_Inout_ PSRWLOCK SRWLock);  
void ReleaseSRWLockExclusive (_Inout_ PSRWLOCK SRWLock);
```

SRW locks store very little state, so their flexibility is limited:

- A shared lock owner cannot directly upgrade its lock to an exclusive one. It must first release its shared lock and then contend for the exclusive lock.
- An exclusive owner cannot recursively acquire a lock; this causes a deadlock.
- There is no guarantee that the first thread acquiring a lock is the first to receive it. As the documentation states: “SRW locks are neither fair nor FIFO”.

Assuming these limitations are acceptable, performance gains are possible if most operations on the data are read rather than write.

RAII Wrappers

As with critical sections, it's convenient to have RAII wrappers for SRWLOCKS. Here are three classes, one for wrapping SRWLOCK and the others for acquiring/releasing:

```
class ReaderWriterLock : public SRWLOCK {
public:
    ReaderWriterLock();
    ReaderWriterLock(const ReaderWriterLock&) = delete;
    ReaderWriterLock& operator=(const ReaderWriterLock&) = delete;

    void LockShared();
    void UnlockShared();

    void LockExclusive();
    void UnlockExclusive();
};

struct AutoReaderWriterLockExclusive {
    AutoReaderWriterLockExclusive(SRWLOCK& lock);
    ~AutoReaderWriterLockExclusive();

private:
    SRWLOCK& _lock;
};

struct AutoReaderWriterLockShared {
    AutoReaderWriterLockShared(SRWLOCK& lock);
    ~AutoReaderWriterLockShared();

private:
    SRWLOCK& _lock;
};
```

The implementation is fairly straightforward:

```
ReaderWriterLock::ReaderWriterLock() {
    ::InitializeSRWLock(this);
}

void ReaderWriterLock::LockShared() {
    ::AcquireSRWLockShared(this);
}

void ReaderWriterLock::UnlockShared() {
    ::ReleaseSRWLockShared(this);
}

void ReaderWriterLock::LockExclusive() {
    ::AcquireSRWLockExclusive(this);
}

void ReaderWriterLock::UnlockExclusive() {
    ::ReleaseSRWLockExclusive(this);
}

AutoReaderWriterLockExclusive::AutoReaderWriterLockExclusive(SRWLOCK& lock)
    : _lock(lock) {
    ::AcquireSRWLockExclusive(&_amp;_lock);
}

AutoReaderWriterLockExclusive::~AutoReaderWriterLockExclusive() {
    ::ReleaseSRWLockExclusive(&_amp;_lock);
}

AutoReaderWriterLockShared::AutoReaderWriterLockShared(SRWLOCK& lock)
    : _lock(lock) {
    ::AcquireSRWLockShared(&_amp;_lock);
}

AutoReaderWriterLockShared::~AutoReaderWriterLockShared() {
    ::ReleaseSRWLockShared(&_amp;_lock);
}
```

These wrappers are part of the *Threadinghelpers* project.

MD5 Calculator 2

With the MD5 Calculator, we can replace some of the critical sections for SRW locks to potentially improve concurrency, as multiple read operations may be happening at the same time. For example, the hash cache use of a critical section can be replaced with an SRWLOCK:

```
class HashCache {
public:
    HashCache();

    bool Add(PCWSTR path, const Hash& hash);
    const Hash Get(PCWSTR path) const;
    bool Remove(PCWSTR path);
    void Clear();

private:
    mutable ReaderWriterLock _lock;
    std::unordered_map<std::wstring, Hash> _cache;
};
```

And the implementation:

```
bool HashCache::Add(PCWSTR path, const Hash& hash) {
    AutoReaderWriterLockExclusive locker(_lock);
    auto it = _cache.find(path);
    if (it == _cache.end()) {
        _cache.insert({ path, hash });
        return true;
    }
    return false;
}
```

```

const Hash HashCache::Get(PCWSTR path) const {
    AutoReaderWriterLockShared locker(_lock);
    auto it = _cache.find(path);
    return it == _cache.end() ? Hash() : it->second;
}

bool HashCache::Remove(PCWSTR path) {
    AutoReaderWriterLockExclusive locker(_lock);
    auto it = _cache.find(path);
    if (it != _cache.end()) {
        _cache.erase(it);
        return true;
    }
    return false;
}

void HashCache::Clear() {
    AutoReaderWriterLockExclusive locker(_lock);
    _cache.clear();
}

```

Similar modifications can be made for the CView class.

The above changes can be found in the *MD5Calculator2* project.

Lastly, SRW locks support *Try* variants for acquiring the locks:

```

BOOLEAN TryAcquireSRWLockExclusive (_Inout_ PSRWLOCK SRWLock);

BOOLEAN TryAcquireSRWLockShared    (_Inout_ PSRWLOCK SRWLock);

```

These functions return TRUE if the specified lock is acquired, and FALSE otherwise. If it is acquired, the normal associated *Release* function must be eventually called.

Condition Variables

Condition variables are another synchronization mechanism, providing the capability to wait on a critical section or SRW lock until some condition occurs. A classic example of using condition variables is a producer/consumer scenario. Suppose some threads produce data items and place them in a queue. Each thread does whatever work is needed to produce the items. At the same time, other threads act as consumers - each removing an item from the queue and processes it in some way (figure 7-11).

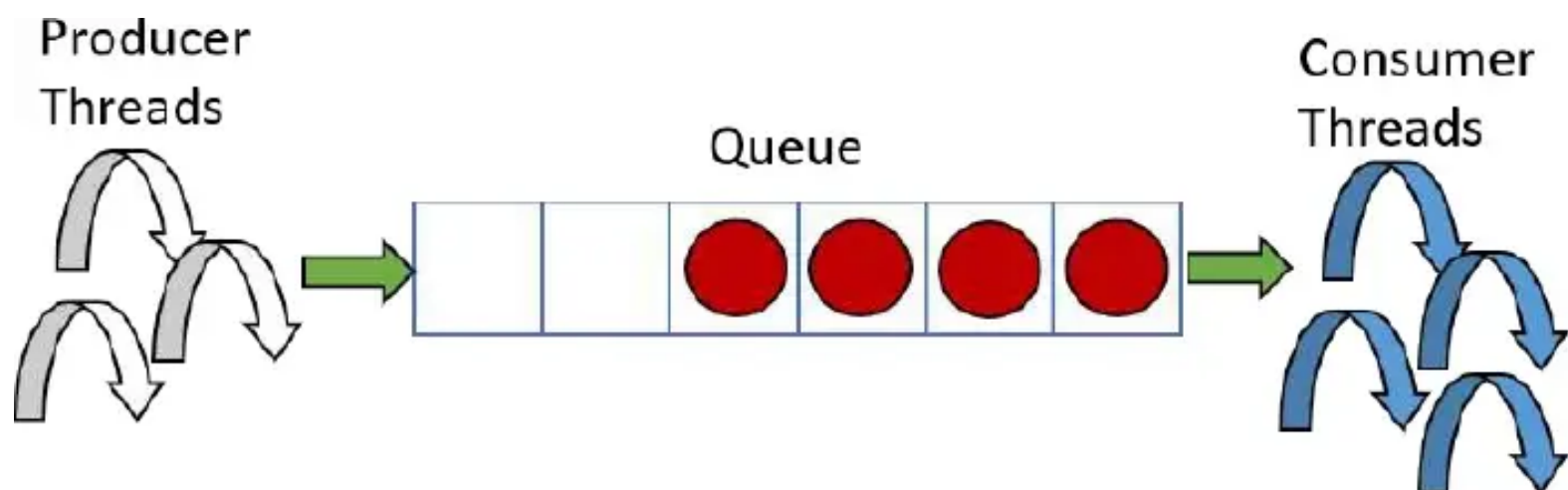


Figure 7-11: Producer/consumer

If items are produced faster than consumers can process, then the queue is non-empty, and consumers continue working. On the other hand, if consumer threads process all items, they should go into a wait state until new items are produced, in which case they should be awoken. This is exactly the behavior provided by condition variables. Consumer threads that have nothing to do (the queue is empty) should not spin, checking periodically if the queue becomes non-empty, because this consumes CPU cycles for no good reason. Condition variables allow an efficient wait (that consumes no CPU) until threads are awakened (typically by producer threads).

A condition variable is represented by a `CONDITION_VARIABLE` opaque structure, very similar to `SRWLOCK`. It must be initialized by calling `InitializeConditionVariable`:

```
void InitializeConditionVariable(_Out_ PCONDITION_VARIABLE ConditionVariable);
```

As with `SRWLOCK`, a static initialization is possible by setting the `CONDITION_VARIABLE` to `CONDITION_VARIABLE_INIT`.

A condition variable is always associated with a critical section or `SRW` lock. When a

thread needs to wait until a condition variable is signaled, it first must acquire the critical section/SRW lock and then call the associated sleep function:

```
BOOL SleepConditionVariableCS(  
    _Inout_ PCONDITION_VARIABLE ConditionVariable,  
    _Inout_ PCRITICAL_SECTION CriticalSection,  
    _In_ DWORD dwMilliseconds);
```

```
BOOL SleepConditionVariableSRW(  
    _Inout_ PCONDITION_VARIABLE ConditionVariable,  
    _Inout_ PSRWLOCK SRWLock,  
    _In_ DWORD dwMilliseconds,  
    _In_ ULONG Flags);
```

The thread calling one of the above Sleep* functions must have first acquired the associated synchronization object exactly once. The function releases the synchronization object and waits on the condition variable, atomically. While waiting, the thread may be woken by calling one of the wake functions on the condition variable:

```
VOID WakeConditionVariable    (_Inout_ PCONDITION_VARIABLE ConditionVariable);
```

```
VOID WakeAllConditionVariable (_Inout_ PCONDITION_VARIABLE ConditionVariable);
```

WakeConditionVariable wakes a single thread (no guarantee on which thread it is if multiple threads are sleeping on the condition variable), while WakeAllConditionVariable wakes all threads waiting on the condition variable.

Once woken, the thread re-acquires the synchronization object and continues execution. At this time, the thread should recheck the condition for which it was waiting, and if not satisfied, call the Sleep* function again. This can happen as another thread might have woken up just before this one and did some work that made the condition false again. The operations of such a thread are illustrated in figure 7-12 (using a critical section).

For SRW locks, the `Flags` parameter indicates if this should be an exclusive acquire or not. Passing zero means exclusive access, while passing `CONDITION_VARIABLE_LOCKMODE_SHARED` means shared access.

The Queue Demo Application

The *Queue Demo* application demonstrates the use of a condition variable to wake consumer threads accessing a shared queue with producer threads. Once launched, it allows selecting the number of producer and consumer threads (figure 7-13).

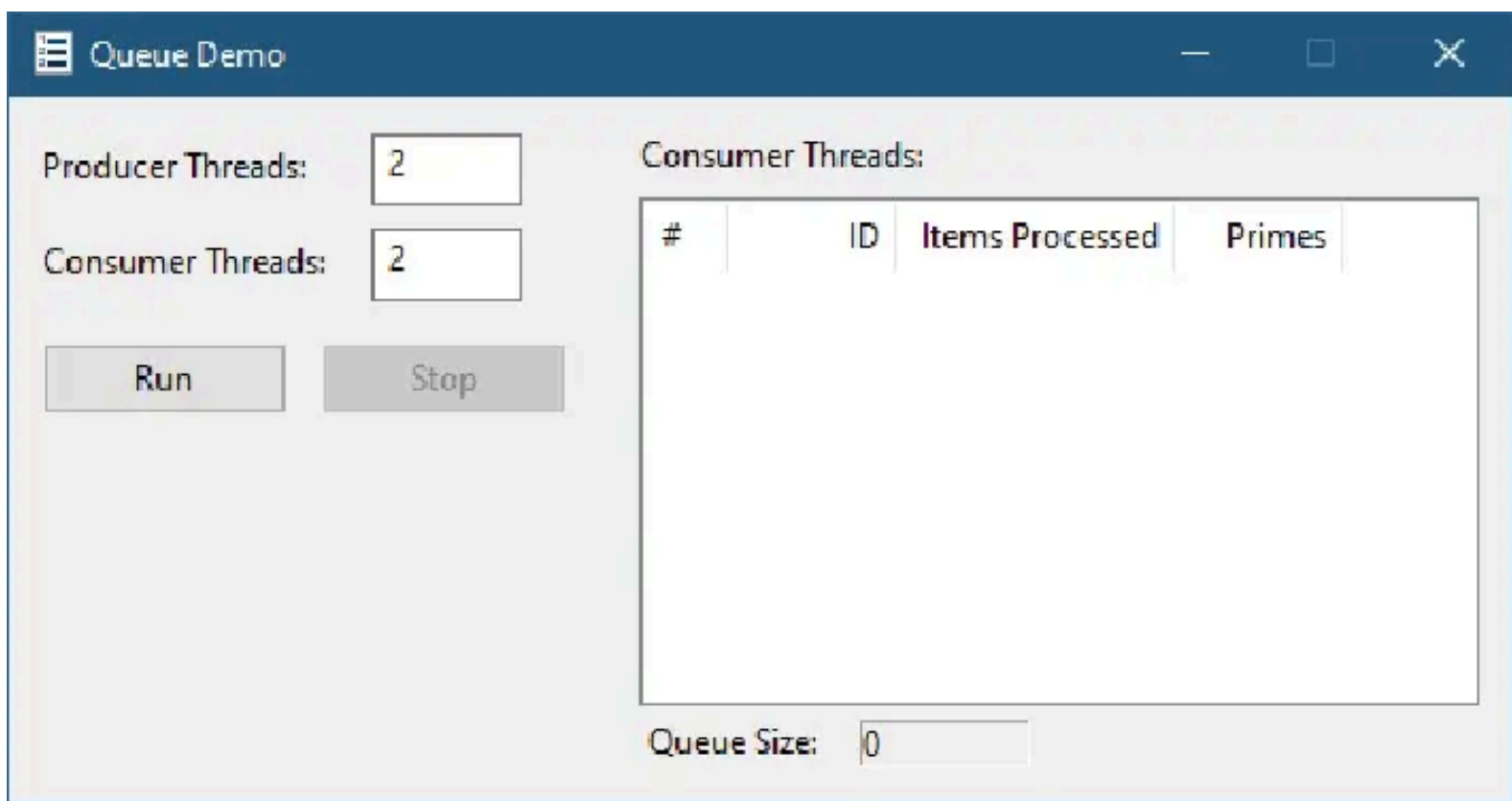


Figure 7-13: The *Queue Demo* application

Click the *Run* button to start the action. Producer threads produce items (which are numbers) and push them into a queue. Consumer threads pop items from the queue and check if the numbers are prime or not. If the queue is empty, a consumer thread sleeps on a condition variable until woken up by a producer thread. The current queue size is shown at the bottom and is updated periodically.

If the producer threads produce more items than the consumer threads can handle, the queue size will increase, as the consumers try to catch up. Clicking *Stop* stops the producers, allowing the consumers to catch up and drain the queue. If, on the other hand, the consumers are “quicker” (perhaps because there are many of them), the queue size will be mostly zero, as consumers are quick enough to pick up any new item and process it before a new one

appears in the queue. While working, some statistics are displayed for the consumer threads (figure 7-14). You'll have to "play" with the thread counts to get interesting behavior.

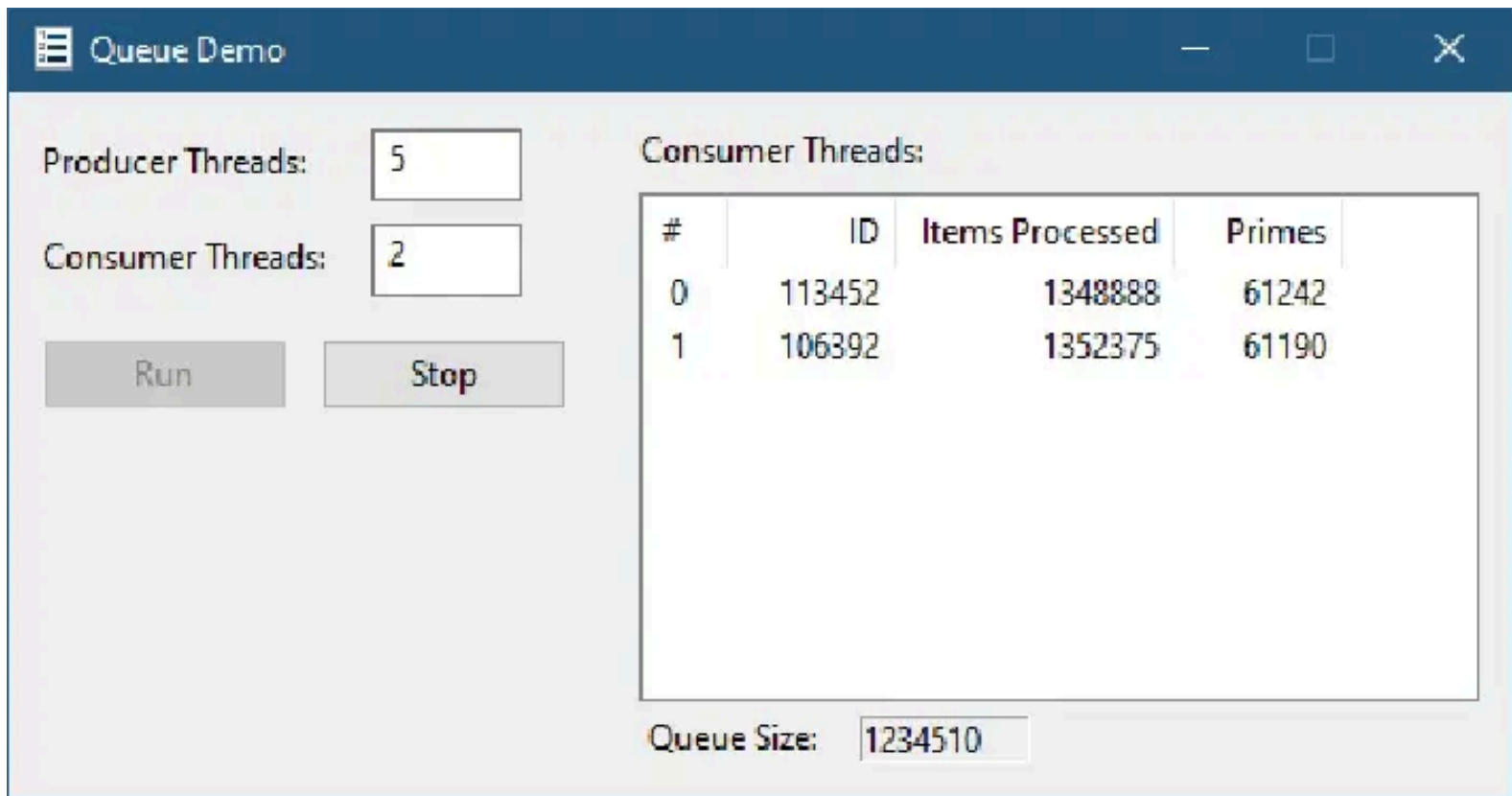


Figure 7-14: The *Queue Demo* application at work

The `CMainDlg` class defines the following nested types:

```
struct ConsumerThreadData {
    unsigned ItemsProcessed{ 0 };
    unsigned Primes{ 0 };
    wil::unique_handle hThread;
};

struct WorkItem {
    unsigned Data;
    bool IsPrime;
};
```

`ConsumerThreadData` is the data structure manipulated by a consumer thread. There is one such object per consumer thread. It stores a handle to the thread, and the count of items processed and prime numbers found. Each work item stored in the queue is made up of a the number to determine if it's prime and a result (which is not used in the application directly).

Based on these structures and the application's requirements, the following data members are stored:

```
std::queue<WorkItem> m_Queue;    // the queue
CriticalSection m_QueueLock;    // the critical section protecting the queue
CONDITION_VARIABLE m_QueueCondVar;
std::vector<wil::unique_handle> m_ProducerThreads;
std::vector<ConsumerThreadData> m_ConsumerThreads;
wil::unique_handle m_hAbortEvent;
static CMainDlg* m_pThis;        // simplifies access to this
```

Producer threads just store their handles, but they could potentially store more state, similar to the consumer threads. The `CriticalSection` class is the wrapper from the *ThreadingHelpers* project to simplify working with it. The `m_hAbortEvent` is an event kernel object handle that is used to signal producer and consumer threads to stop running. Event kernel objects are discussed in detail in the next chapter. As an alternative for this application, a volatile boolean variable could have been used instead. Finally, The `m_pThis` static member is used to refer to the only dialog instance to simplify accessing instance methods for thread functions.

The `CMainDlg::OnInitDialog` function performs single initialization of the dialog box in terms of controls, but also initializes `m_pThis` and the abort event:

```
LRESULT CMainDlg::OnInitDialog(UINT, WPARAM, LPARAM, BOOL&) {
    m_pThis = this;

    m_hAbortEvent.reset(::CreateEvent(nullptr, TRUE, FALSE, nullptr));
    //...
```

Clicking the *Run* button causes a call to `OnRun` which just calls `Run` to do the actual work. The function starts by getting the number of consumer and producer threads and performs some sanity checks:

```

void CMainDlg::Run() {
    int consumers = GetDlgItemInt(IDC_CONSUMERS);
    if (consumers < 1 || consumers > 64) {
        DisplayError(L"Consumer threads must be between 1 and 64");
        return;
    }
    int producers = GetDlgItemInt(IDC_PRODUCERS);
    if (producers < 1 || producers > 64) {
        DisplayError(L"Producer threads must be between 1 and 64");
        return;
    }
}

```

There is nothing special about the number 64 in the above code. Larger numbers can easily be selected if desired.

Next, some initialization should be done for this run:

```

bool abort = false;
::ResetEvent(m_hAbortEvent.get());
::InitializeConditionVariable(&m_QueueCondVar);
m_ThreadList.DeleteAllItems();

```

The abort event is reset and the condition variable is initialized. The list view showing the consumer threads is cleared of any existing items. Now it's time to create the consumer threads:

```

m_ConsumerThreads.clear();
m_ConsumerThreads.reserve(consumers);

for (int i = 0; i < consumers; i++) {
    ConsumerThreadData data;
    data.hThread.reset(::CreateThread(nullptr, 0, [](auto p) {
        return m_pThis->ConsumerThread(PtrToLong(p));
    }, LongToPtr(i), 0, nullptr));
    if (!data.hThread) {

```

```

        abort = true;
        break;
    }
    m_ConsumerThreads.push_back(std::move(data));
}
if (abort) {
    ::SetEvent(m_hAbortEvent.get());
    return;
}

```

Each consumer thread is created with a normal `CreateThread`, pointing the thread function to the instance function `ConsumerThread` with a value indicating the index of the consumer thread in the consumer thread array.



You might think it would be easier to just pass the thread function a pointer to the `ConsumerThreadData` instance (`data` in the code above). This will lead to a crash or corruption, because the data is on the stack and then will be copied to the vector (and so moved to the heap) making the pointer a garbage one. In this case, I decided to pass in the index as it's not going to change.

If, for some reason, thread creation fails, the loop is aborted and the event is set to cause all previously created producer threads to abort.

Next, producer threads are created in a similar fashion:

```

m_ProducerThreads.clear();
m_ProducerThreads.reserve(producers);

for (int i = 0; i < producers; i++) {
    wil::unique_handle hThread(::CreateThread(nullptr, 0, [] (auto p) {
        return m_pThis->ProducerThread();
    }, this, 0, nullptr));
    if (!hThread) {
        DisplayError(L"Failed to create producer thread. Aborting");
        abort = true;
        break;
    }
}
}

```

```

if (abort) {
    ::SetEvent(m_hAbortEvent.get());
    return;
}

```

Producer threads call the `ProducerThread` instance function. Contrary to consumer threads, these don't require any special context, since they just generate pseudo-random numbers. The last part of the `Run` function is adding the consumer threads basic information to the list view and start a simple timer used to periodically update the queue size:

```

CString text;
for (int i = 0; i < (int)m_ConsumerThreads.size(); i++) {
    const auto& t = m_ConsumerThreads[i];
    text.Format(L"%2d", i);
    int n = m_ThreadList.InsertItem(i, text);
    m_ThreadList.SetItemText(n, 1,
        std::to_wstring(::GetThreadId(t.hThread.get())).c_str());
}

```

```

GetDlgItem(IDC_RUN).EnableWindow(FALSE);
GetDlgItem(IDC_STOP).EnableWindow(TRUE);

```

```

SetTimer(1, 500, nullptr);

```

Here is a producer thread's code:

```

DWORD CMainDlg::ProducerThread() {
    for (;;) {
        if (::WaitForSingleObject(m_hAbortEvent.get(), 0) == WAIT_OBJECT_0)
            break;

        WorkItem item;
        item.IsPrime = false;
        LARGE_INTEGER li;
        ::QueryPerformanceCounter(&li);
        item.Data = li.LowPart;
        {

```

```

        AutoCriticalSection locker(m_QueueLock);
        m_Queue.push(item);
    }
    ::WakeConditionVariable(&m_QueueCondVar);

    // sleep a little bit from time to time
    if ((item.Data & 0x7f) == 0)
        ::Sleep(1);
}
return 0;
}

```

The code uses an infinite loop which is only broken out of if the abort event is signaled. Then a `WorkItem` instance is prepared, and the number generated uses the low 32 bit of whatever `QueryPerformanceCounter` returns. This choice is completely arbitrary in this example. Next, the thread acquires the critical section to prevent synchronization issues on the work items queue, since it's accessed concurrently by producers and consumers (and even the UI thread may need access). The queue itself is the standard C++ `std::queue<>` class, but any other queue implementation will do.

Once a new item is added to the queue, the thread signals the condition variable to wake up a thread that is waiting on it by calling `WakeConditionVariable`. The last bit of code before looping to push the next item is a possible sleep to delay the thread a bit.

The consumer threads' code is shown below:

```

DWORD CMainDlg::ConsumerThread(int index) {
    auto& data = m_ConsumerThreads[index];
    auto tick = ::GetTickCount64();

    for (;;) {
        WorkItem value;
        {
            bool abort = false;
            AutoCriticalSection locker(m_QueueLock);
            while (m_Queue.empty()) {
                if (::WaitForSingleObject(m_hAbortEvent.get(), 0) == WAIT_OBJECT_
T_0) {
                    abort = true;
                    break;

```

```

        }
        ::SleepConditionVariableCS(&m_QueueCondVar, &m_QueueLock, INFIN\
ITE);
    }
    if (abort)
        break;

    ATLASSTERT(!m_Queue.empty());
    value = m_Queue.front();
    m_Queue.pop();
}

{
    // do the actual work

    bool isPrime = IsPrime(value.Data);

    if (isPrime) {
        value.IsPrime = true;
        ::InterlockedIncrement(&data.Primes);
    }
    ::InterlockedIncrement(&data.ItemsProcessed);
}
auto current = ::GetTickCount64();
if (current - tick > 600) {
    PostMessage(WM_UPDATE_THREAD, index);
    tick = current;
}
}

PostMessage(WM_UPDATE_THREAD, index);

return 0;
}

```

The function starts by getting a reference to the data structure for this consumer thread. Then an infinite loop is constructed. The critical section is acquired for the queue and then an inner while loop is entered if the queue is empty. If it's empty, the thread has nothing to do, so it calls `SleepConditionVariableCS` to enter a wait state until awakened by another thread

using condition variable. Before waiting, it releases the critical section. When it wakes up (because a producer called `WakeConditionVariable`), it automatically acquires the critical section again (see figure 7-12), and must recheck the condition (queue empty), because it's possible that another consumer thread woke up a bit earlier and popped the last item from the queue. Also, condition variables may be prone to spurious wake ups, which is another reason to recheck the condition.

If the queue is not empty, the consumer can go ahead and remove the first item in the queue (at this time the critical section is still held) by calling `m_Queue.front()` to get the item and `m_Queue.pop()` to drop it from the queue.

The critical section is then released (scope ended with `AutoCriticalSection`) and then actual work on the item is performed by calling the helper `IsPrime` function. The relevant counters maintained by this thread are incremented if needed. The increments are performed with `InterlockedIncrement` because the UI thread accesses these values possibly concurrently. Finally, a message is posted to the window to update the statistics of this thread every 600 milliseconds or so.

The `WM_UPDATE_THREAD` application-defined message receives the consumer thread index and updates the number of items processed and the number of primes calculated:

```
LRESULT CMainDlg::OnUpdateThread(UINT, WPARAM index, LPARAM, BOOL&) {
    auto& data = m_ConsumerThreads[index];
    int n = (int)index;

    CString text;
    text.Format(L"%u", ::InterlockedAdd((LONG*)&data.ItemsProcessed, 0));
    m_ThreadList.SetItemText(n, 2, text);
    text.Format(L"%u", ::InterlockedAdd((LONG*)&data.Primes, 0));
    m_ThreadList.SetItemText(n, 3, text);

    return 0;
}
```

Accessing the counters may be done concurrently with respect to the consumer thread, so to prevent possible torn reads, `InterlockedAdd` with zero is used to mitigate that. Reading the counters directly would probably be fine too, but it may depend on the values' alignment in memory and on the target processor, so it's better to be safe than sorry.

The other UI update is the periodic queue size update by using a timer:

```

LRESULT CMainDlg::OnTimer(UINT, WPARAM id, LPARAM, BOOL&) {
    if (id == 1) {
        size_t size;
        {
            AutoCriticalSection locker(m_QueueLock);
            size = m_Queue.size();
        }
        SetDlgItemInt(IDC_QUEUE_SIZE, (unsigned)size, FALSE);
    }
    return 0;
}

```

The timer ID used is 1, checked by the `if` statement. Any access to the queue should be done under the protection of the critical section, and then the queue size is read before updating the UI.

Lastly, clicking the *Stop* button calls the `OnStop` function which simply calls `Stop`:

```

void CMainDlg::Stop() {
    // signal threads to abort

    ::SetEvent(m_hAbortEvent.get());
    ::WakeAllConditionVariable(&m_QueueCondVar);

    // update UI

    GetDlgItem(IDC_RUN).EnableWindow(TRUE);
    GetDlgItem(IDC_STOP).EnableWindow(FALSE);
}

```

The function sets the abort event to cause all producers to exit their infinite loop. The condition variable is used to wake all consumer threads, so they can drain the queue if any items are left in it.



Write a RAII wrapper for condition variables.

Change the *Queue Demo* application to use an SRW lock instead of a critical section.

Waiting on Address

Windows 8 and Server 2012 adds another synchronization mechanism, that allows a thread to wait efficiently until the value at some address changes to a desired value. Then it can wake up and proceed with its work. It's certainly possible to use other synchronization mechanisms to achieve a similar effect, such as using a condition variable, but waiting on address is more efficient and is not prone to deadlocks since no critical sections (or other software synchronization primitives) are used directly.

A thread can enter a wait state until a certain value appears on a “monitored” data by calling `WaitOnAddress`:

```
BOOL WaitOnAddress(  
    _In_ volatile VOID* Address,  
    _In_ PVOID CompareAddress,  
    _In_ SIZE_T AddressSize,  
    _In_opt_ DWORD dwMilliseconds);
```



The functions in this section require linking against the *synchronization.lib* import library.

The function checks if the value at `*Address` is the same as in `*CompareAddress`. If they are different, the call returns immediately with a value of `TRUE`. Otherwise, the thread enters a wait state. The values size to compare is specified in the `AddressSize` parameter, and it must be 1, 2, 4 or 8. The last parameter indicates the time to wait, which can be `INFINITE` to wait however long it takes.

Internally, the kernel keeps waited on addresses in a hash table, keyed by the address.

Some other thread may change the value in `*Address`. Unfortunately, this does not automatically cause the waiting thread to wake. Instead, the thread that made the change must call one of the “wake” functions:

```
VOID WakeByAddressSingle (_In_ PVOID Address);
```

```
VOID WakeByAddressAll (_In_ PVOID Address);
```

With the former function, Any threads waiting on the specified address are woken, while the latter wakes just one thread. Spurious wakes are also possible with this mechanism, so the woken thread should recheck if the value is indeed the expected one. If not, the thread should go back to waiting by calling `WaitOnAddress` again (typically done with a loop).

A typical code could look something like this:

```
DWORD undesiredValue = 0;
```

```
DWORD actualValue = 0;
```

```
void Thread1() {
    // set undesiredValue as appropriate

    while(actualValue == undesiredValue) {
        ::WaitOnAddress(&actualValue, &undesiredValue, sizeof(DWORD), INFINITE);
    }

    // actualValue != undesiredValue
}

void Thread2() {
    //...
    actualValue++;
    ::WakeByAddressSingle(&actualValue);
}
```

Synchronization Barriers

Another synchronization primitive introduced in Windows 8 is a *synchronization barrier*. This object allows synchronizing threads that need to get to a certain point in their work before they all can continue. For example, suppose there are several parts of a system, each of which needs to be initialized in two phases, before the main application code can continue. One simple way to accomplish that is to call each initialization function sequentially:

```
void RunApp() {  
    // phase 1  
  
    InitSubsystem1();  
    InitSubsystem2();  
    InitSubsystem3();  
    InitSubsystem4();  
  
    // phase 2  
  
    InitSubsystem1Phase2();  
    InitSubsystem2Phase2();  
    InitSubsystem3Phase2();  
    InitSubsystem4Phase2();  
  
    // go ahead and run main application code...  
}
```

This works, but if each of the initializations can be done concurrently, so that each initialization is carried out by a different thread. Each thread must not continue to phase 2 initialization until all other threads are done with phase 1. It is, of course, possible to implement such a scheme by using a combination of other synchronization primitives, but a synchronization barrier already exists for this kind of purpose.

A synchronization barrier is represented by the `SYNCHRONIZATION_BARRIER` opaque structure that must be initialized with `InitializeSynchronizationBarrier`:

```
BOOL InitializeSynchronizationBarrier(  
    _Out_ LPSYNCHRONIZATION_BARRIER lpBarrier,  
    _In_ LONG lTotalThreads,  
    _In_ LONG lSpinCount);
```

`lTotalThreads` is the total number of threads that need to reach the barrier before they all can continue. The `lSpinCount` parameter allows setting a spin count for the threads arriving at the barrier before entering a wait state (if the barrier is not yet released). A value of `-1` sets a default spin.

The documentation states that the default spin is 2000. However, as far as I can tell the spin count is not currently used.

Once initialized, threads that need to wait at the barrier call `EnterSynchronizationBarrier`:

```
BOOL EnterSynchronizationBarrier(
    _Inout_ LPSYNCHRONIZATION_BARRIER lpBarrier,
    _In_ DWORD dwFlags);
```

Specifying the `SYNCHRONIZATION_BARRIER_FLAGS_SPIN_ONLY` flag causes the thread to spin until the barrier is released. This should only be used if the anticipated time for barrier release is small. The opposite flag, `SYNCHRONIZATION_BARRIER_FLAGS_BLOCK_ONLY`, specifies that no spinning should be done if the barrier is not released yet and then thread should enter a wait state. The last flag, `SYNCHRONIZATION_BARRIER_FLAGS_NO_DELETE` is a possible optimization that tells the API to skip some synchronization required when deleting the barrier. If specified, all threads entering the barrier must specify this flag.

The function returns `TRUE` for a single thread only once the barrier is released, and `FALSE` for all other threads. In the previously described scenario, here is one of the initialization functions running in a separate thread:

```
DWORD WINAPI InitSubSystem1(PVOID p) {
    auto barrier = (PSYNCHRONIZATION_BARRIER)p;

    // phase 1
    printf("Subsystem 1: Starting phase 1 initialization (TID: %u)...\n",
        ::GetCurrentThreadId());
    // do work...
    printf("Subsystem 1: Ended phase 1 initialization...\n");
    ::EnterSynchronizationBarrier(barrier, 0);

    printf("Subsystem 1: Starting phase 2 initialization...\n");
    // do work
    printf("Subsystem 1: Ended phase 2 initialization...\n");
}
```

```

    return 0;
}

```

After phase 1 initialization is complete, `EnterSynchronizationBarrier` is called to wait until all other threads complete their phase 1 initialization. The main function can be written like so:

```

SYNCHRONIZATION_BARRIER sb;
InitializeSynchronizationBarrier(&sb, 4, -1);

LPTHREAD_START_ROUTINE functions[] = {
    InitSubSystem1, InitSubSystem2, InitSubSystem3, InitSubSystem4
};

printf("System initialization started\n");
HANDLE hThread[4];
int i = 0;
for (auto f : functions) {
    hThread[i++] = ::CreateThread(nullptr, 0, f, &sb, 0, nullptr);
}
::WaitForMultipleObjects(_countof(hThread), hThread, TRUE, INFINITE);
printf("System initialization complete\n");
// close thread handles...

```

Running this piece of code produces output like the following:

```

System initialization started
Subsystem 1: Starting phase 1 initialization (TID: 79480)...
Subsystem 2: Starting phase 1 initialization (TID: 104836)...
Subsystem 3: Starting phase 1 initialization (TID: 32556)...
Subsystem 4: Starting phase 1 initialization (TID: 86268)...
Subsystem 2: Ended phase 1 initialization...
Subsystem 3: Ended phase 1 initialization...
Subsystem 1: Ended phase 1 initialization...
Subsystem 4: Ended phase 1 initialization...
Subsystem 4: Starting phase 2 initialization...
Subsystem 3: Starting phase 2 initialization...
Subsystem 1: Starting phase 2 initialization...

```

```
Subsystem 2: Starting phase 2 initialization...
Subsystem 3: Ended phase 2 initialization...
Subsystem 1: Ended phase 2 initialization...
Subsystem 4: Ended phase 2 initialization...
Subsystem 2: Ended phase 2 initialization...
System initialization complete
```

Finally, a synchronization barrier should be deleted with `DeleteSynchronizationBarrier`:

```
BOOL DeleteSynchronizationBarrier(_Inout_ LPSYNCHRONIZATION_BARRIER lpBarrier);
```

It's OK to call `DeleteSynchronizationBarrier` immediately after calling `EnterSynchronizationBarrier` because the function waits until all threads reached the barrier before being deleted, unless all threads use the flag `SYNCHRONIZATION_BARRIER_FLAGS_NO_DELETE` so that the delete function does not make that guarantee. This could be useful if the barrier is never deleted.

What About the C++ Standard Library?

Similar to the same-named section from chapter 5, the C++ standard library provides synchronization primitives that can be used as alternatives to the Windows API, especially for cross-platform code. As usual, customization on these objects is very limited (if any). Examples include:

- `std::mutex` that acts like a critical section without the support for recursive acquisition.
- `std::recursive_mutex` that acts just like a critical section (supports recursive acquisition).
- `std::shared_mutex` that is similar to an SRW lock.
- `std::condition_variable` is a conditional variable equivalent.
- Others

Clearly, some things may be missing from C++, such as waiting on address and synchronization barriers. However, these could be added to a future standard. In any case, all the C++ standard library types work within the same process only. There is no way to use these across processes.

Exercises

1. Create a thread-safe implementation of a stack data structure. If a pop operation cannot succeed, block the thread until data is available (use a condition variable).

Summary

In this chapter, we looked at various synchronization mechanisms available through the Windows API. Common to all these is the ability to synchronize in some sense between threads in the same process. In the next chapter, we'll extend the synchronization primitives available by leveraging kernel objects that can also synchronize between threads in different processes.